



Université Grenoble Alpes
Laboratoire de Conception et d'Intégration des Systèmes

Implementing multiple control methods

on a 2-DOF Helicopter

Advisor

Prof. Dr. Eng. Florin Stoican

Co-advisors

Assoc. Prof. Dr. Eng. Ionela Prodan

Asst. Prof. Dr. Eng. Daniel Ioan

Asst. Prof. Eng. Bogdan Gheorghe

Student

David-Mihai Bicu

Valence, 2026

Contents

1	Introduction	2
2	Quanser Aero 2	3
2.1	Introduction	3
2.2	System schematic	4
3	Mathematical model	6
3.1	Nonlinear model	6
3.2	Linear model	8
3.3	Properties of the calculated linear model	12
4	Classical Control Methods based on the linear model	14
4.1	Pole Placement - MATLAB implementation	14
4.2	Linear Quadratic Regulator - MATLAB Implementation	26
4.3	Linear Quadratic Integrator - MATLAB Implementation	33
4.4	PP controller based on a modified linear model	41
4.5	Conclusions	45
5	Predictive Control Methods	46
5.1	Introduction to Predictive Control	46
5.2	Linear Model-based Predictive Control	47
5.3	Nonlinear Model-based Predictive Control	48
5.4	MPC Implementation	52
	Appendices	55
A	Discrete model	55
	Bibliography	57

1 Introduction

2 Quanser Aero 2

2.1 Introduction

The Quanser Aero 2 experiment is a laboratory setup designed to study helicopter dynamics and control. It is a coupled electromechanical system where the rotors, that control the yaw and pitch motions, interact, creating cross-axis torques.



Figure 1: Quanser Aero 2 experiment model taken from [1]

It can be configured in 3 different modes:

- **Half-Quadrotor:** the pitch axis is locked, but the yaw axis is free. Both rotors are horizontal.
- **1-DOF VTOL:** the pitch axis is free, but the yaw axis is locked. Both rotors are horizontal.
- **2-DOF Helicopter:** both axis are unlocked and the rear rotor is vertical.

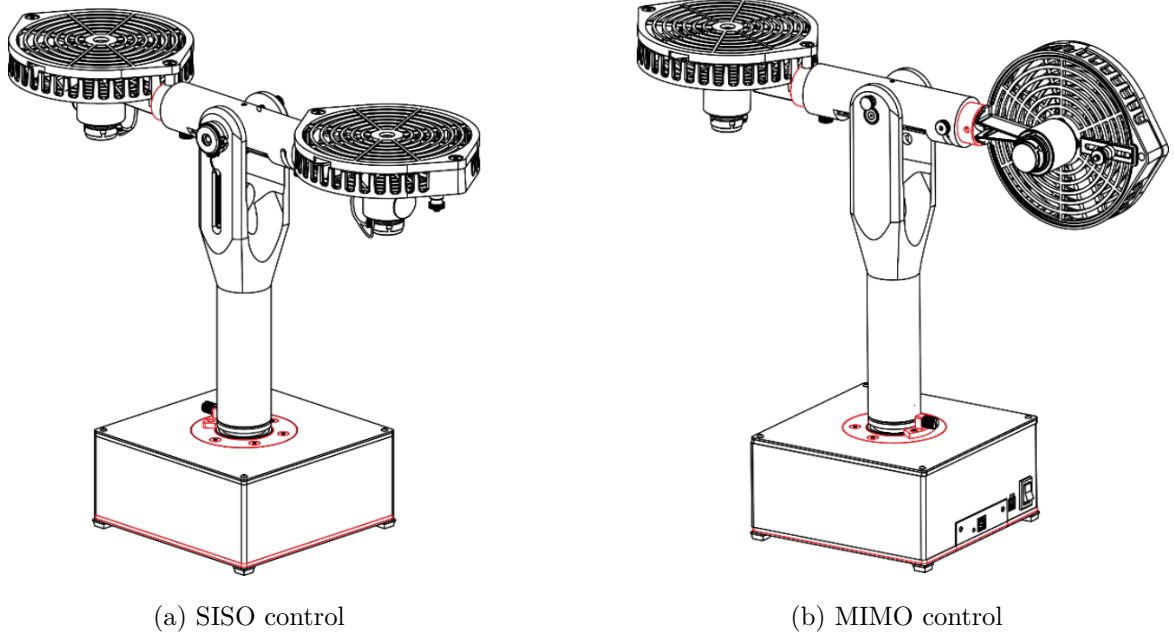


Figure 2: SISO and MIMO control configurations

The first 2 are used for Single-Input, Single-Output (SISO) control; while the 2-DOF Helicopter is used for Multiple-Input, Multiple Output (MIMO) control.

On this setup, we will apply on the MIMO system multiple control methods with the purpose of following certain references on both the pitch and yaw axes.

2.2 System schematic

According to [1], it is equipped with an external DC power supply which provides 24.0 V power at up to 2.71 A for the sensors and motors. This indicates that the command from the PC/Microcontroller will be designed to fit in the interval $[-24.0 \text{ V}, +24.0 \text{ V}]$.

Both the rotors use direct 18 V DC brushed motors that are connected to two eight-vane counter-rotating 3D-printed propellers those apply a strong cross-torque during the thrust. The command is then sent to the encoder that sends it to the Data Acquisition block that communicates directly with the Simulink interface on a PC (via the QFLEX2 USB interface) or with an microcontroller (via the QFLEX2 Embedded interface) and with the pitch/yaw encoders that read both the angular position, as well as the angular speed.

There are three types of encoders:

- pitch encoders: single-ended optical shaft encoders, with 2880 counts per revolution in quadrature mode (720 lines/rev)
- motor encoders: single-ended optical shaft encoders, with 2048 counts per revolution in quadrature mode (512 lines/rev)
- yaw encoder: optical encoder, with 4096 counts per revolution in quadrature mode (1024 lines/rev)

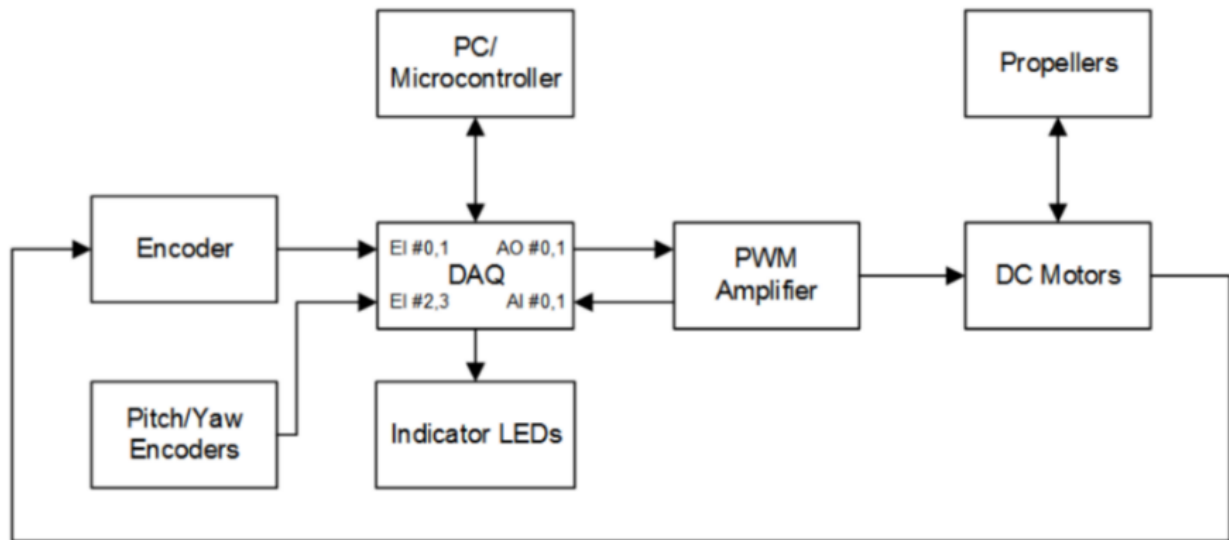


Figure 3: Interaction between the Quanser Aero 2 system components

The system includes an integrated Inertial Measurement Unit (IMU) mounted on the Aero core board, which allows for real-time measurements of the angular position and velocity on both the pitch and yaw axes. The accelerometer has a range of $\pm 8g$ range on 16 bits, while the gyroscope has a range of ± 500 deg/s on 16 bits.

3 Mathematical model

3.1 Nonlinear model

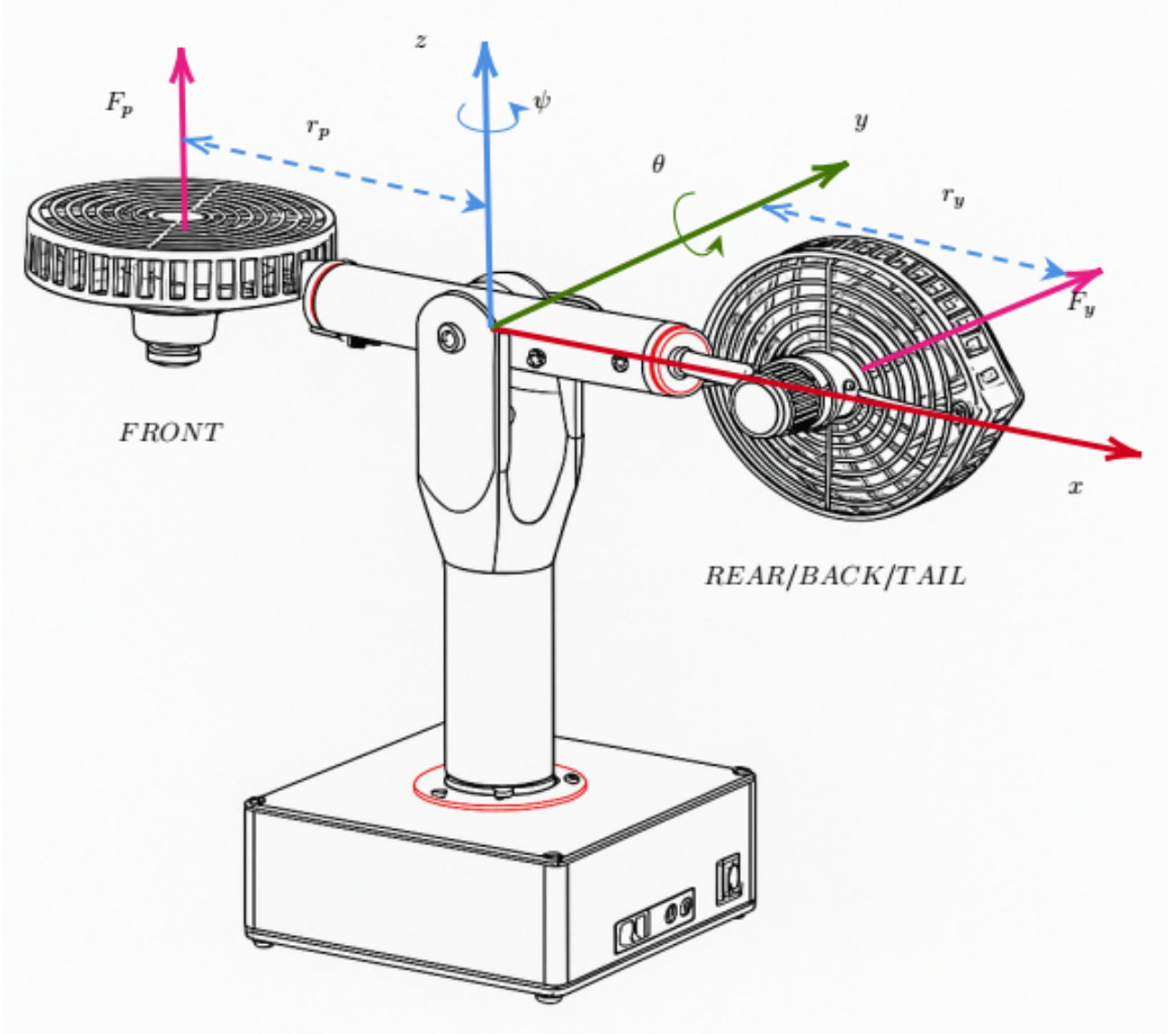


Figure 4: Simple free body diagram of the Quanser Aero 2

The free-body diagram of the Quanser Aero 2 is presented in Figure 4. The following conventions are used for the modeling:

- The helicopter is horizontal and parallel with the ground when the pitch $\theta = 0$ rad
- The pitch angle increases positively ($\dot{\theta}(t) > 0$ rad/s), when the front rotor is moved upward and the body rotates counter-clockwise (CCW) around the Y axis.
- The yaw angle increases positively ($\dot{\psi}(t) > 0$ rad/s), when the body rotates CCW around the Z axis.
- Pitch increases $\dot{\theta}(t) > 0$ rad/s, when the front rotor voltage is positive $V_p > 0$ V
- Yaw increases $\dot{\psi}(t) > 0$ rad/s, when the tail rotor voltage is positive $V_y > 0$ V

In order to obtain the nonlinear model, we have to look at the equations obtained from Newton's Second Law applied on the circular motions around the pitch and yaw axis.

For the motion around the pitch axis, the only torques that rotate the system in the same direction as $\dot{\theta}$ are the ones generated from the rotor engines, which are directly proportional to the input voltage of each engine, which sits at the distance $D_t = r_p = r_y$ from the pivot:

$$\tau_p = D_t (K_{pp}V_p + K_{py}V_y)$$

The opposing torques are generated by the dampening coefficient ($\tau_v = -D_p\dot{\theta}$), by the system's stiffness around the pitch axis ($\tau_{sp} = -K_{sp}\theta$) and the Coriolis torque ($-m\dot{\psi}^2 l^2 \sin(\theta) \cos(\theta)$). The inertial moment J_p is determined by the inertial moment of the propellers ($J_{prop} = m_{prop}r_{prop}^2$) and the one of the body ($J_{body} = \frac{m_{body}L_{body}^2}{12}$): $J_p = J_{body} + 2J_{prop}$.

The motion equation around the pitch axis is:

$$(J_p + ml^2) \ddot{\theta} = D_t K_{pp} V_p + D_t K_{py} V_y - D_p \dot{\theta} - mgl \cos(\theta) - K_{sp} \theta - m\dot{\psi}^2 l^2 \sin(\theta) \cos(\theta) \quad (1)$$

The motion equation around the yaw axis is calculated similarly, except the fact that the helicopter doesn't have stiffness around the yaw axis and we have to consider the inertial moment of the yolk, which is approximated with a cylinder ($J_{york} = \frac{m_{york}r_{york}^2}{2} \implies J_y = J_{body} + 2J_{prop} + J_{york}$), which leads to this equation:

$$(J_y + ml^2 \cos^2(\theta)) \ddot{\psi} = D_t K_{yp} V_p + D_t K_{yy} V_y - D_y \dot{\psi} + ml^2 \dot{\psi} \dot{\theta} \sin(2\theta) \quad (2)$$

And we obtain this nonlinear model:

$$\begin{aligned} (J_p + ml^2) \ddot{\theta} &= D_t K_{pp} V_p + D_t K_{py} V_y - D_p \dot{\theta} - K_{sp} \theta - mgl \cos(\theta) - m\dot{\psi}^2 l^2 \sin(\theta) \cos(\theta) \\ (J_y + ml^2 \cos^2(\theta)) \ddot{\psi} &= D_t K_{yp} V_p + D_t K_{yy} V_y - D_y \dot{\psi} + ml^2 \dot{\psi} \dot{\theta} \sin(2\theta) \end{aligned} \quad (3)$$

with the states:

- θ - pitch angular position, measured in rad, but presented in deg in graphics
- ψ - yaw angular position, measured in rad, but presented in deg in graphics
- $\dot{\theta}$ - pitch angular speed, measured in rad/s
- $\dot{\psi}$ - yaw angular speed, measured in rad/s

and the inputs:

- V_p - input voltage for the front rotor motor, measured in V
- V_y - input voltage for the rear rotor motor, measured in V

both being constrained between $[-24 \text{ V}, +24 \text{ V}]$

3.2 Linear model

The system is stable at any yaw angle, but only when the body is parallel with the ground, which is equivalent with the pitch angle equal to 0.

For obtaining the linear model around the operating point $(\theta, \psi, \dot{\theta}, \dot{\psi}) = \mathbf{0}$, we have to isolate the input voltages V_p and V_y , because it is already linear, in order to study the free evolution of the pitch and yaw movements:

$$\begin{cases} F_\theta(\theta, \dot{\theta}, \dot{\psi}) = -D_p \dot{\theta} - K_{sp} \theta - mgl \cos(\theta) - m\dot{\psi}^2 l^2 \sin(\theta) \cos(\theta) \\ F_\psi(\theta, \dot{\theta}, \dot{\psi}) = -D_y \dot{\psi} + m\dot{\psi} \dot{\theta} \sin(2\theta) \end{cases}$$

We linearize these functions around the operating point by using the Taylor expansion:

$$\begin{cases} F_\theta(\theta, \dot{\theta}, \dot{\psi}) \approx F_\theta(0, 0, 0) + \left. \frac{\partial F_\theta}{\partial \theta} \right|_{(0,0,0)} \theta + \left. \frac{\partial F_\theta}{\partial \dot{\theta}} \right|_{(0,0,0)} \dot{\theta} + \left. \frac{\partial F_\theta}{\partial \dot{\psi}} \right|_{(0,0,0)} \dot{\psi} \\ F_\psi(\theta, \dot{\theta}, \dot{\psi}) \approx F_\psi(0, 0, 0) + \left. \frac{\partial F_\psi}{\partial \theta} \right|_{(0,0,0)} \theta + \left. \frac{\partial F_\psi}{\partial \dot{\theta}} \right|_{(0,0,0)} \dot{\theta} + \left. \frac{\partial F_\psi}{\partial \dot{\psi}} \right|_{(0,0,0)} \dot{\psi} \end{cases}$$

which leads to the linearized approximations:

Because the $F_\theta(0, 0, 0) = -mgl$ and $F_\psi(0, 0, 0) = 0$, we have to look at the stationary point for the linear model:

$$\begin{cases} 0 = D_t K_{pp} V_{p0} + D_t K_{py} V_{y0} - mgl \implies mgl = D_t K_{pp} V_{p0} + D_t K_{py} V_{y0} \\ 0 = D_t K_{yp} V_{p0} + D_t K_{yy} V_{y0} \end{cases} \quad (4)$$

From which we obtain the linear system:

$$\begin{bmatrix} mgl \\ \frac{D_t}{0} \end{bmatrix} = \begin{bmatrix} K_{pp} & K_{py} \\ K_{yp} & K_{yy} \end{bmatrix} \begin{bmatrix} V_{p0} \\ V_{y0} \end{bmatrix} \quad (5)$$

Because V_{p0} is outside the physical constraints, we have to presume that $V_{y0} = 0$ V, because the front motor could compensate the torque generated by the gravity :

$$mgl = D_t K_{pp} V_{p0} \implies V_{p0} = \frac{mgl}{D_t K_{pp}} = 46.8733 \text{ V} > 24 \text{ V}$$

Because V_{p0} sits outside the physical constraints, we would suppose that $V_{p0} = V_{y0} = 0$ V

$$\begin{aligned} F_\theta(\theta, \dot{\theta}, \dot{\psi}) + mgl &\triangleq \Delta F_\theta(\theta, \dot{\theta}, \dot{\psi}) \approx -K_{sp} \theta - D_p \dot{\theta} \\ F_\psi(\theta, \dot{\theta}, \dot{\psi}) &\approx -D_y \dot{\psi} \end{aligned} \quad (6)$$

and the linear system:

$$\begin{aligned} \ddot{\theta} &= -\frac{K_{sp}}{J_p + ml^2} \theta - \frac{D_p}{J_p + ml^2} \dot{\theta} + \frac{K_{pp}}{J_p + ml^2} V_p + \frac{K_{py}}{J_p + ml^2} V_y \\ \ddot{\psi} &= -\frac{D_y}{J_y + ml^2} \dot{\psi} + \frac{K_{yp}}{J_y + ml^2} V_p + \frac{K_{yy}}{J_y + ml^2} V_y \end{aligned} \quad (7)$$

which looks like this in state-space, considering the state vector

$$x = [\theta \quad \psi \quad \dot{\theta} \quad \dot{\psi}]^T \in \mathbb{R}^4$$

the input vector

$$u = [V_p \ V_y]^T \in \mathbb{R}^2$$

and the output vector $y = [\theta \ \psi]^T \in \mathbb{R}^2$:

$$\begin{cases} \begin{bmatrix} \dot{\theta} \\ \dot{\psi} \\ \ddot{\theta} \\ \ddot{\psi} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{K_{sp}}{J_p + ml^2} & 0 & -\frac{D_p}{J_p + ml^2} & 0 \\ 0 & 0 & 0 & -\frac{D_y}{J_y + ml^2} \end{bmatrix} \begin{bmatrix} \theta \\ \psi \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ \frac{D_t K_{pp}}{J_p + ml^2} & \frac{D_t K_{py}}{J_p + ml^2} \\ \frac{D_t K_{yp}}{J_y + ml^2} & \frac{D_t K_{yy}}{J_y + ml^2} \end{bmatrix} \begin{bmatrix} V_p \\ V_y \end{bmatrix} \\ \begin{bmatrix} \theta \\ \psi \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \theta \\ \psi \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_p \\ V_y \end{bmatrix} \end{cases} \quad (8)$$

Parameters used: The data sheet [1] and the lab instructions [2] offer a list of all the physical constraints of the system, most of them being identified experimentally:

- $m = 1.07$ kg – total body mass
- $l = 0.00242$ m – Distance from each engine to the pivot
- $D_t = 0.16743$ m – Distance from each engine to the pivot
- $K_{sp} = 0.00744$ N · m/rad – Stiffness coefficient about the pitch axis
- $D_p = 0.00199$ N · m · s/rad – Viscous damping coefficient about the pitch axis
- $D_y = 0.00192$ N · m · s/rad – Viscous damping coefficient about the yaw axis
- $K_{pp} = 0.00321$ N/V – Torque thrust gain from the pitch rotor
- $K_{yy} = 0.00610$ N/V – Torque thrust gain from the yaw rotor
- $K_{py} = 0.00137$ N/V – Cross-torque thrust gain acting on the pitch axis from the yaw rotor
- $K_{yp} = -0.00319$ N/V – Cross-torque thrust gain acting on the yaw axis from the pitch rotor

The ones that were calculated were the moments of inertia around the pitch axis (J_p) and the yaw axis (J_y), which were calculated by approximating the shapes with discs and cylinders, as emphasized right before the nonlinear model equations 1 and 2¹:

- $J_p = 0.0232$ kg · m² – Total moment of inertia about the pitch axis
- $J_y = 0.0238$ kg · m² – Total moment of inertia about the yaw axis

¹In the lab instructions, both values are given in g · mm², but we converted them in kg · m² to match all unit measures for the model

Linear State-Space Model:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -0.3208 & 0 & -0.0858 & 0 \\ 0 & 0 & 0 & -0.0806 \end{bmatrix} \in \mathbb{R}^{4 \times 4}$$

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0.0232 & 0.0099 \\ -0.0224 & 0.0429 \end{bmatrix} \in \mathbb{R}^{4 \times 2}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 4}$$

$$D = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2}$$

For this model, I calculated the optimal commands by solving the eq. (5), where I obtained these results:

$$V_{p0} = 38.3205 \text{ V} > 24 \text{ V}$$

$$V_{y0} = 20.0390 \text{ V}$$

We also calculated the linear model that would go in the state $x^* = [0 \ 0 \ 0 \ 0]$ in MATLAB by using a nonlinear model implemented in Simulink by using an MATLAB Function

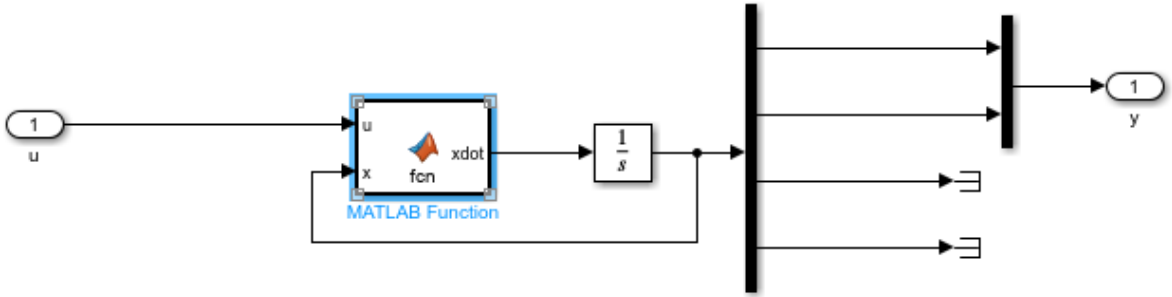


Figure 5: Simulink scheme for the non-linear model

and the MATLAB Function:

```

1 function xdot = fcn(u, x)
2
3 theta = x(1);
4 psi = x(2);
5 theta_dot = x(3);
6 psi_dot = x(4);
7
8 Vp = u(1);
9 Vy = u(2);
10
11 % listing all the parameters from the lab indicators and the manual
12
13 theta_ddot = (Dt*Kpp*Vp + ...
14               Dt*Kpy*Vy - ...
15               Mb*g*Dm*cos(theta) - ...

```

```

16      Dp*theta_dot - ...
17      Ksp*theta - ...
18      Mb*psi_dot^2*Dm^2*sin(theta)*cos(theta)) / (Jp + Mb*Dm^2);
19 psi_ddot = (Dt*Kyp*Vp + ...
20      Dt*Kyy*Vy - ...
21      Dy*psi_dot + ...
22      Mb*Dm^2*psi_dot*theta_dot*sin(2*theta)) / (Jy + Mb*Dm^2*cos(
23      theta)^2);
24 xdot = [theta_dot, psi_dot, theta_ddot, psi_ddot]';

```

which, through the commands `trim` and `linmod`, I obtained this stationary point for the model if I start in the state $x_0 = [0 \ 0 \ 0 \ 0]^T$ and the results are identical.

Before we study the properties of the linear system, we have to check if it responds the same as the real system and as the QUARC Simulation at the same input.

For this, we applied both to the QUARC Simulator, the real plant, the nonlinear model and the calculated linear model these inputs:

$$V_p(t) = 5 \cdot \mathbf{1}(t - 1) \text{ [V]}$$

$$V_y(t) = 7 \cdot \mathbf{1}(t - 1) \text{ [V]}$$

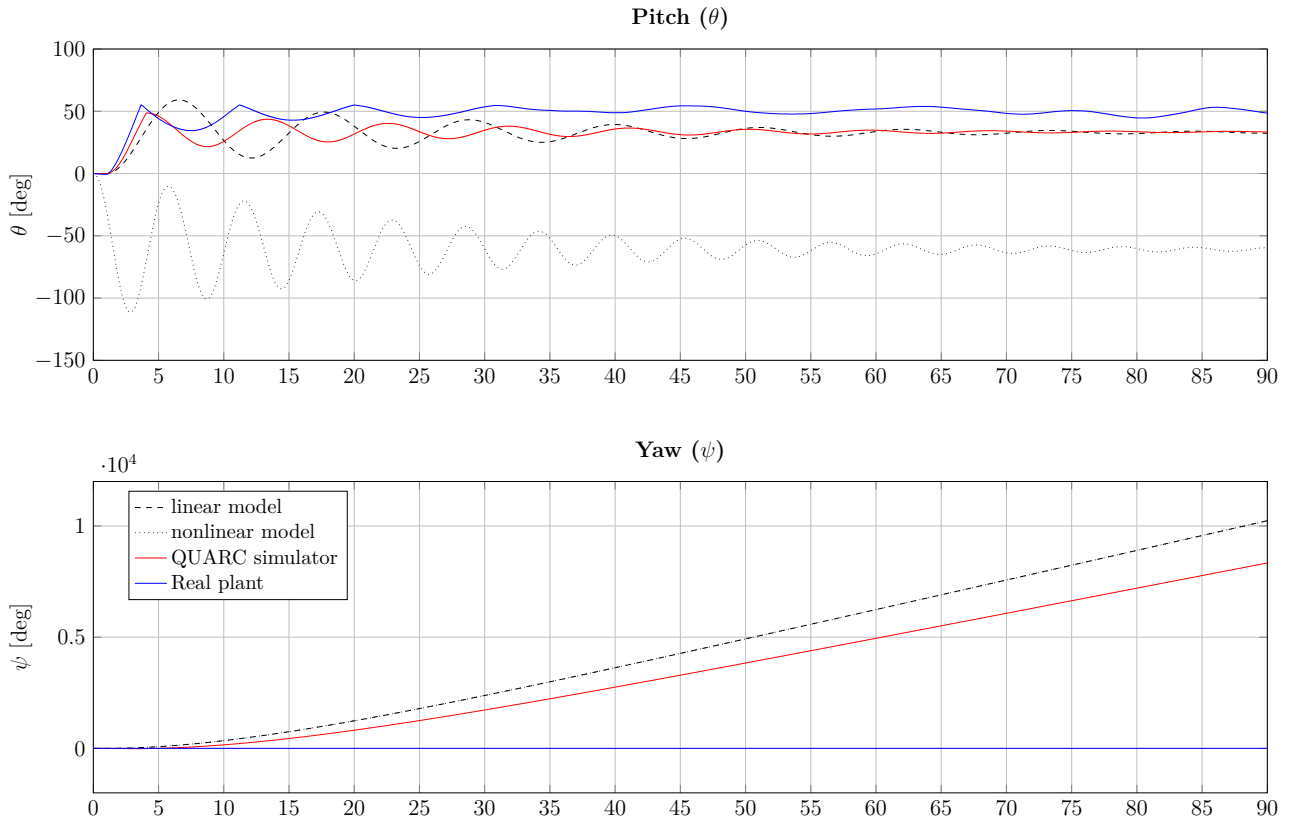


Figure 6: Simulation results of all the 4 models

In fig. 6, we observe that the pitch evolution is similar between the QUARC simulation and the linear model, the main difference being caused by the saturation of the pitch at the value $\theta_s \approx \pm 51 \text{ deg} \approx \pm 0.89 \text{ rad}$. Moreover, due to the stiffness in the motion, switches the trajectory instantly at the moment of reaching the saturation value, which causes the anticipation in its response, while both models stabilize around the same value. Also, the yaw response of both models captures the response of a ramp signal with a slight difference in the steady-state yaw

rate: $\dot{\psi} \approx 131.78 \text{ deg/s} \approx 2.3 \text{ rad/s}$ for the linear model, while for the QUARC Simulator is slightly lower at $\dot{\psi} \approx 111.73 \text{ deg/s} \approx 1.95 \text{ rad/s}$, which might be caused by the ignorance of the friction torques inside the Simulator that oppose the yaw motion. Also, the results on the real plant show that there is no motion in the yaw axis ($\dot{\psi}(t) = 0 \text{ rad/s}, \forall t \geq 0$), which forces the pitch to oscillate around the saturation value due to the lack of the Coriolis torque that opposes the pitch motion. This phenomenon might be caused by a friction torque in the yaw axis that is bigger than the torque created by the input voltages. Because of this, there will be significant differences between the real plant and the QUARC Simulator when we implement the control methods, because the real plant needs inputs with higher magnitude than the simulator in order to achieve a similar output, while both systems have the same input constraints.

The main problem with these results is that the nonlinear model is not constrained like the simulator or the real model, hence why it is able to set its inputs outside the physical constraints and build the linearization around these equilibrium points that do not fit the constraints. While the linear model stabilizes around the same point as the nonlinear model, implementing multiple control methods around these 2 models might cause unexpected behaviour of the QUARC simulator or, worse, the real plant, which already needs higher inputs than the simulator for a similar output.

Because of these results, we will use the calculated linear model eq. (7) for the implementation of the PP, LQR and LQI controllers, but not before studying its properties.

3.3 Properties of the calculated linear model

Open-loop stability For the stability of the system we have to look at the eigenvalues of the A matrix, which are the roots of the characteristic polynomial:

$$\det(\lambda I_4 - A) = 0 \iff \begin{vmatrix} \lambda & 0 & -1 & 0 \\ 0 & \lambda & 0 & -1 \\ 0.3208 & 0 & \lambda + 0.0858 & 0 \\ 0 & 0 & 0 & \lambda + 0.0806 \end{vmatrix} = 0$$

We notice that the only non-zero element on the last line of the determinant is $\lambda + 0.0806$, which means that the equation is equivalent with

$$(\lambda + 0.0806) \begin{vmatrix} \lambda & 0 & -1 \\ 0 & \lambda & 0 \\ 0.3208 & 0 & \lambda + 0.0858 \end{vmatrix} = 0 \iff (\lambda + 0.0806) [\lambda^2(\lambda + 0.0858) + 0.3208\lambda] = 0$$

We notice that one of the roots is $\lambda_1 = 0$, which leads to this equation:

$$\lambda(\lambda + 0.0806) (\lambda^2 + 0.0858\lambda + 0.3208) = 0$$

with the solutions:

$$\begin{cases} \lambda_1 = 0 \\ \lambda_2 = -0.0806 \\ \lambda_{3,4}^2 + 0.0858\lambda_{3,4} + 0.3208 = 0 \iff \lambda_{3,4} = -0.0429 \pm 0.5648i \end{cases}$$

We observe that all of the eigenvalues are stable, with the exception of $\lambda_1 = 0$, which is caused by the integrator in the yaw motion eq. (2). To better emphasise the integrator in the yaw motion equation, we will extract the transfer functions from the eq. (7), considering zero initial conditions ($\psi(0^-) = \theta(0^-) = 0 \text{ rad}, \dot{\psi}(0^-) = \dot{\theta}(0^-) = 0 \text{ rad/s}$):

$$\begin{cases} s^2 J_p \theta(s) + s D_p \theta(s) + K_{sp} \theta(s) = K_{pp} V_p(s) + K_{py} V_y(s) \\ s^2 J_y \psi(s) + s D_y \psi(s) = K_{yp} V_p(s) + K_{yy} V_y(s) \end{cases} \iff \begin{cases} \theta(s) = T_{pp}(s) V_p(s) + T_{py}(s) V_y(s) \\ \psi(s) = T_{yp}(s) V_p(s) + T_{yy}(s) V_y(s) \end{cases}$$

Where the transfer functions are:

$$T_{pp}(s) = \frac{K_{pp}}{s^2 J_p + s D_p + K_{sp}} \quad T_{py}(s) = \frac{K_{py}}{s^2 J_p + s D_p + K_{sp}}$$

$$T_{yp}(s) = \frac{K_{yp}}{s(s J_y + D_y)} \quad T_{yy}(s) = \frac{K_{yy}}{s(s J_y + D_y)}$$

The integrator element in the transfer functions for the yaw motion $T_{yp}(s)$ and $T_{yy}(s)$ shows that the system is internally stable, but not asymptotically stable and the feedback control is necessary if the system is controllable.

Controllability: Considering that we have $n = 4$ states, the controllability matrix looks like this:

$$\mathcal{R} = [B \quad AB \quad A^2B \quad A^3B]$$

$$\mathcal{R} = \begin{bmatrix} 0 & 0 & 0.0232 & 0.0099 & -0.0020 & -0.0008 & -0.0073 & -0.0031 \\ 0 & 0 & -0.0224 & 0.0429 & 0.0018 & -0.0035 & -0.0001 & 0.0003 \\ 0.0232 & 0.0099 & -0.0020 & -0.0008 & -0.0073 & -0.0031 & 0.0013 & 0.0005 \\ -0.0224 & 0.0429 & 0.0018 & -0.0035 & -0.0001 & 0.0003 & 0.0000 & -0.0000 \end{bmatrix}$$

Considering the fact that $\mathcal{R} \in \mathbb{R}^{4 \times 8}$ is not square, we have determine its rank through its minors. If we calculate the minor formed by the first 4 lines and 4 columns of the controllability matrix, we obtain:

$$\Delta = \begin{vmatrix} 0 & 0 & 0.0232 & 0.0099 \\ 0 & 0 & -0.0224 & 0.0429 \\ 0.0232 & 0.0099 & -0.0020 & -0.0008 \\ -0.0224 & 0.0429 & 0.0018 & -0.0035 \end{vmatrix} \approx 1.4788 \cdot 10^{-6} \neq 0 \implies \text{rank}(\mathcal{R}) = 4 = n$$

Which means the system is controllable.

Observability: The observability matrix looks like this, considering that $n = 4$:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ CA^3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -0.3208 & 0 & -0.0858 & 0 \\ 0 & 0 & 0 & -0.0806 \\ 0.0275 & 0 & -0.3135 & 0 \\ 0 & 0 & 0 & 0.0065 \end{bmatrix} \in \mathbb{R}^{8 \times 4}$$

$\mathcal{O}$ is not a square matrix, so we have to look at its minors in order to determine its rank. We notice that the first 4 lines and first 4 columns form the identity matrix, whose determinant is equal to 1, which means the system is also observable.

Because this state-space realization is both controllable and observable, it means it is minimal and we can configure all 4 eigenvalues in the closed-loop system to reach stability and the performances requirements of tracking a reference with zero steady-state error and minimal overshoot.

4 Classical Control Methods based on the linear model

4.1 Pole Placement - MATLAB implementation

We start the control methods with computing a gain $K \in \mathbb{R}^{2 \times 4}$ that will move the open-loop eigenvalues to the desired poles.

Our objective, inspired from [2], is to track 2 square-wave references with the amplitudes $A_1 = 20$ deg for the pitch, respectively $A_2 = 45$ deg for the yaw, and the frequencies $f_1 = 0.05$ Hz, respectively $f_2 = 0.04$ Hz, while respecting these requirements for the settling phase:

- **pitch motion:** $\sigma \leq 5\%$, $t_p \leq 2.5$ seconds
- **pitch motion:** $\sigma \leq 5\%$, $t_p \leq 3.5$ seconds

Our approach starts from fixing the overshoots σ_1 , σ_2 and the peak times t_{p1} , t_{p2} of a step response, from which we extract the dampening ratios ζ_1 , ζ_2 and the natural frequencies ω_{n1} , ω_{n2} from these formulas:

$$\sigma = 100e^{-\frac{\pi\zeta}{\sqrt{1-\zeta^2}}} \implies \zeta = \frac{\ln\left(\frac{100}{\sigma}\right)}{\sqrt{\pi^2 + \ln^2\left(\frac{100}{\sigma}\right)}}$$

$$t_p = \frac{\pi}{\omega_n \sqrt{1-\zeta^2}} \implies \omega_n = \frac{\pi}{t_p \sqrt{1-\zeta^2}}$$

from which we extract the desired poles:

$$p_{1,2} = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$$

and obtain gain regulator K by calling the function $K = \text{place}(A,B,P)$ in MATLAB. After testing that the placement of the closed-loop poles was successful in MATLAB, we will test the PP regulator K in 3 Simulink files: one with the linear model, one with the nonlinear QUARC simulator and one with the real Quanser Aero 2 (the difference between the Simulator and the real plant are the configured parameters in the HIL Initialize block):

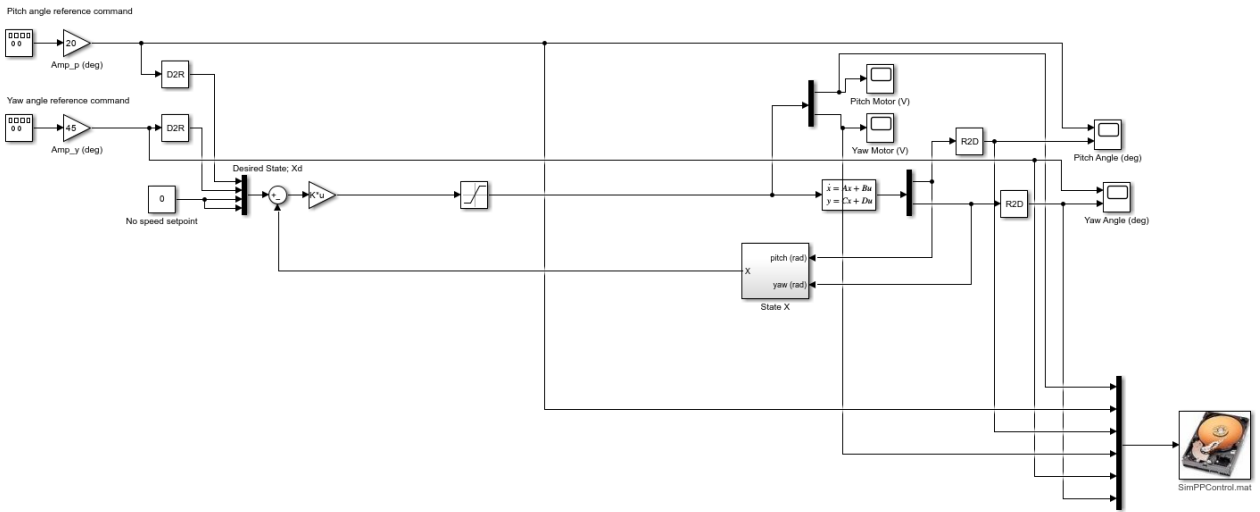


Figure 7: Simulink test scheme on the linear model

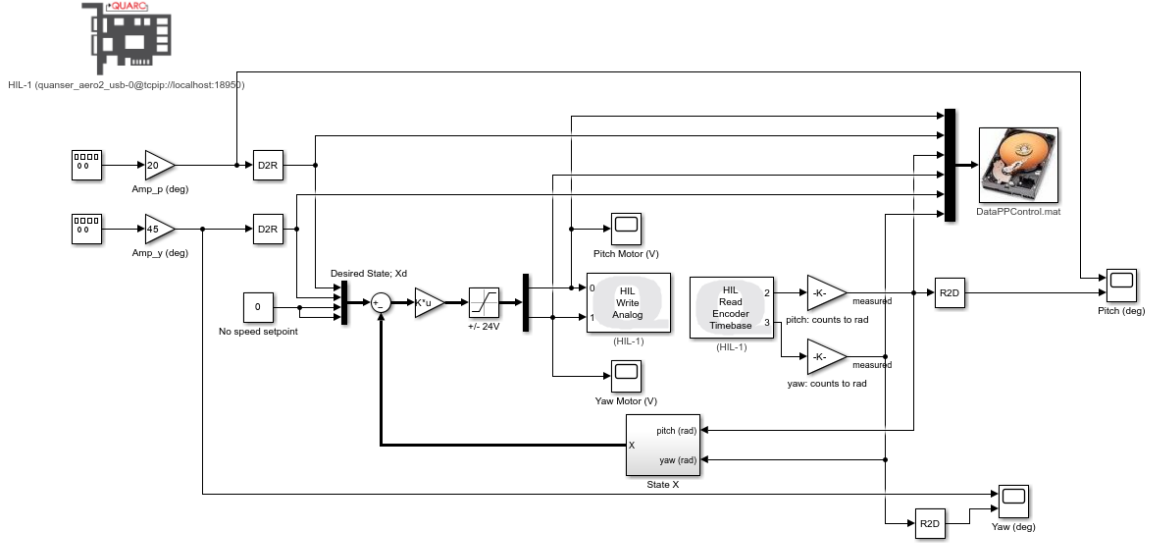


Figure 8: Simulink test scheme on the QUARC Simulator and on the real Quanser Aero 2

We set the following parameters to compute the K gain, because the `place` command doesn't guarantee the exact order of the poles, so I chose for both motions to have identical performance indicators and identical complex-conjugated pairs for the poles:

$$\begin{aligned}\sigma_1 &= 4.5\% & t_{p1} &= 1.5 \text{ sec} \\ \sigma_2 &= 4.5\% & t_{p2} &= 1.5 \text{ sec}\end{aligned}$$

to obtain this set of desired poles:

$$\begin{aligned}p_{1,2} &= -2.0674 \pm 2.0944j \\ p_{3,4} &= -2.0674 \pm 2.0944j\end{aligned}\tag{9}$$

After computing $K = \text{place}(A,B,P)$, we obtained the same eigenvalues of the closed-loop system as the desired poles.

After checking the stability of the closed-loop system, we tested the computed gain on all three models:

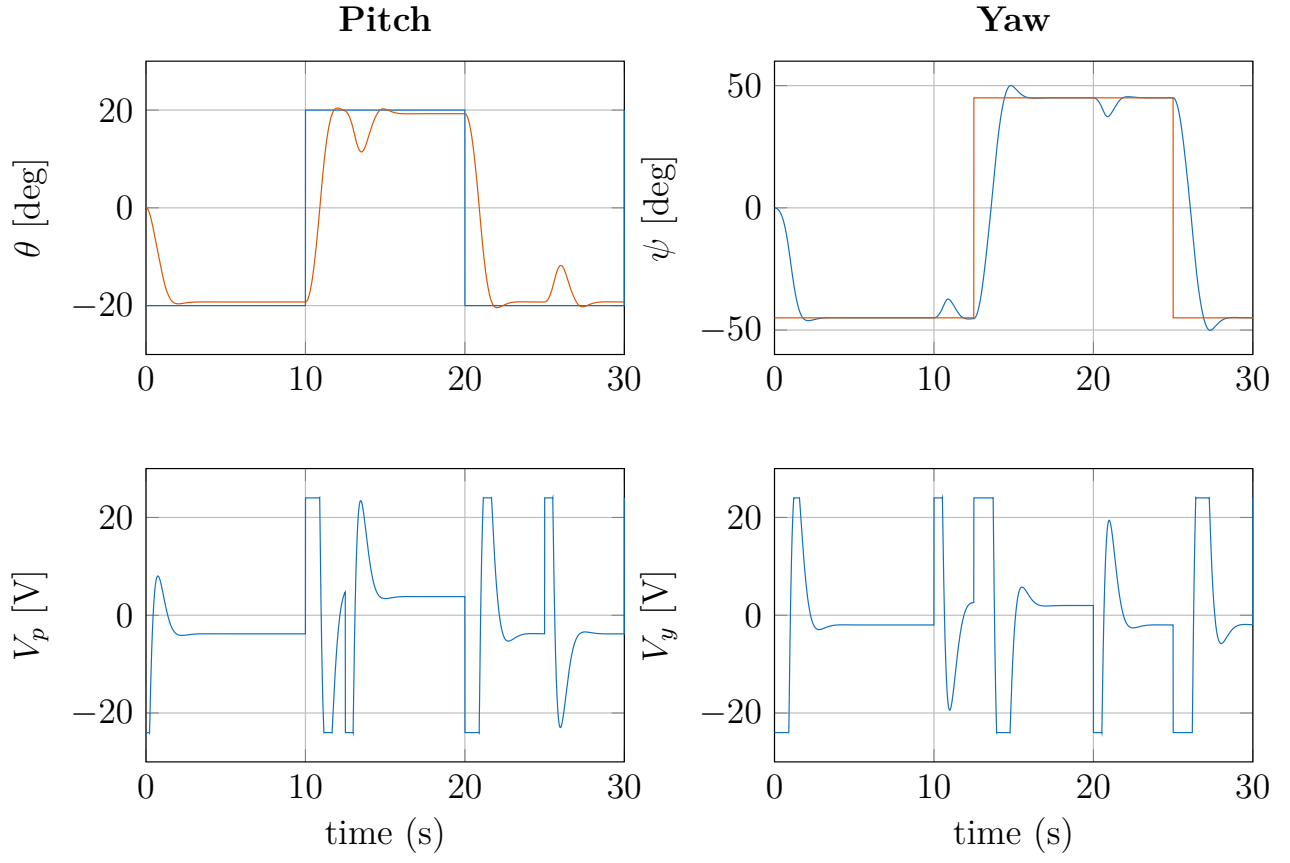


Figure 9: The response of the simulation process with the linear model

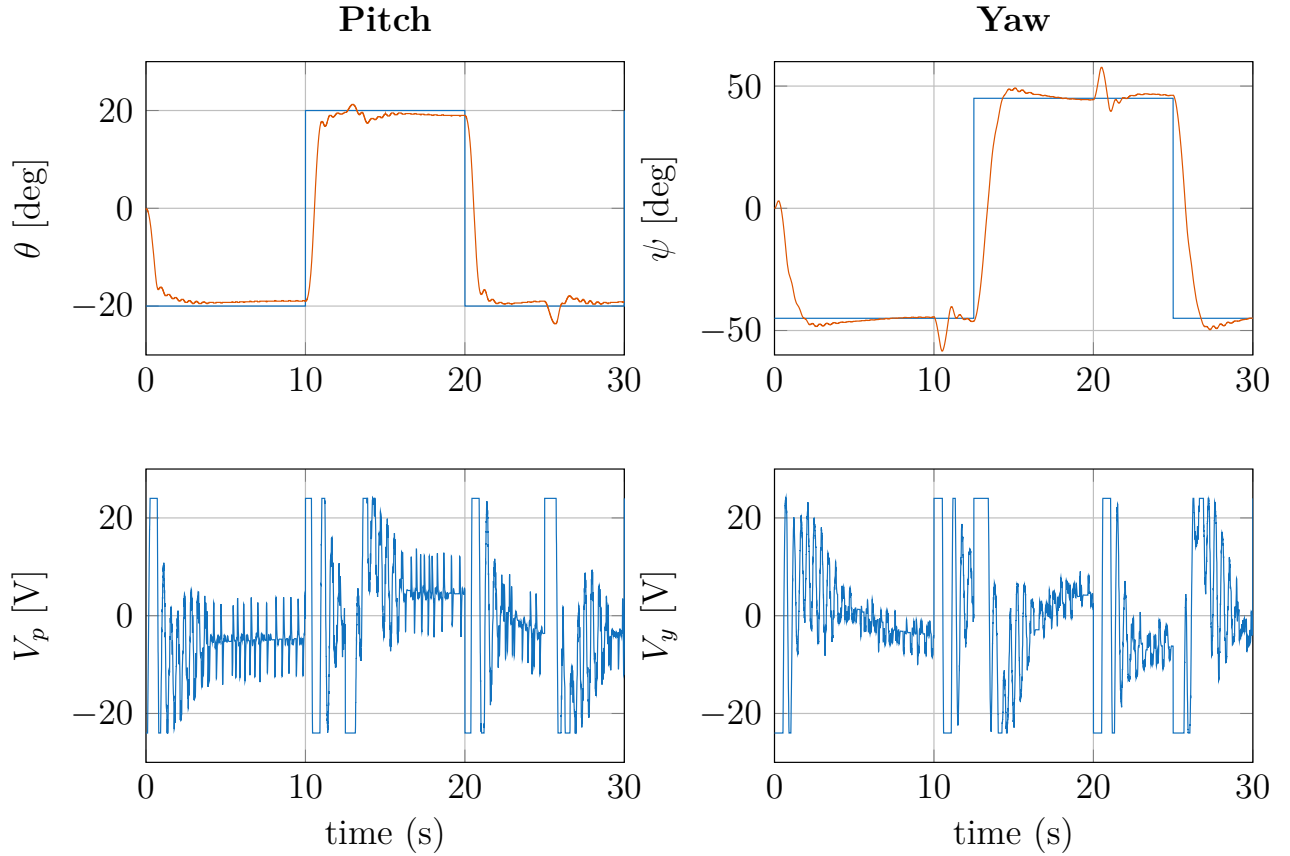


Figure 10: The response of the simulation process with the QUARC Simulator

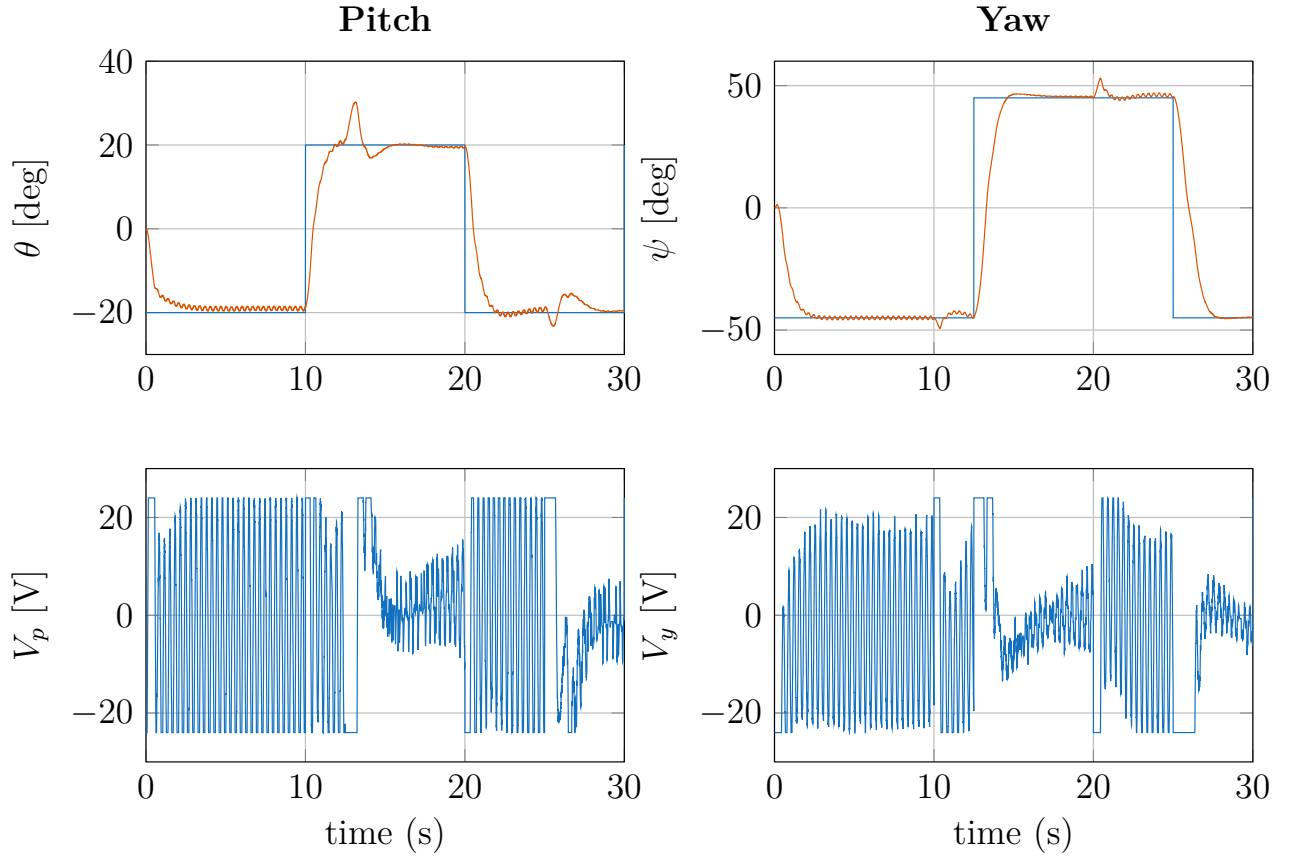


Figure 11: The response of the real plant

Pitch Axis (θ)			
Parameter	Linear Model	Simulator	Real Plant
Relative steady state error (ϵ)	$\approx 3.7\%$	$\approx 5\%$	$\approx 5\%$
Overshoot (σ)	$\approx 5.77\%$	$\approx 11.85\%$	$\approx 4.4\%$
Peak time (t_p)	2.2 s	3 s	2.5 s
Coupling overshoot (σ')	$\approx -39\%$	$\approx +22.36\%$	$\approx +28.13\%$
Yaw Axis (ψ)			
Parameter	Linear Model	Simulator	Real Plant
Relative steady state error (ϵ)	0	1.5%	0
Overshoot (σ)	$\approx 11.25\%$	$\approx 7.4\%$	$\approx 4.87\%$
Peak time (t_p)	2.2 s	2.5 s	2.7s
Coupling overshoot (σ')	$\approx -17.1\%$	$\approx -26.55\%$	$\approx -11.71\%$

Table 1: Performance comparison of the PP Controller across the three tests

In terms of the amplitude of the inputs, the input in the linear model is the smoothest one, as it reaches the saturation state only when one of the 2 reference signals changes its sign. The input in the QUARC Simulator has still a relatively small amplitude, reaching the saturated state under the same conditions as the one in the linear model, but it has high frequency oscillations with a small amplitude. Also, approving our conclusion of testing the models from ????, the command inputs in the real plant have a higher magnitude than ones in the QUARC, resembling an impulse train with the peak points at the saturation state: ± 24 V.

Also, the outputs of both the real plant, as well as the QUARC Simulator present the Gibbs phenomenon, which might be caused by the Coriolis effect, a nonlinear effect ignored in the linearization process.

The input command in both the QUARC Simulator and the real Quanser Aero 2 plant has a high amplitude and high oscillations, which is why we decided to switch the **Saturation Block** in Simulink with a **MATLAB Function** that could approximate the saturation function between U_{min} and U_{max} with a smooth function.

One approach is by building the saturation by using the $G(x) = \tanh x$ function, an elegant way being this transformation²:

$$y(u, m) = \frac{U_{min}G[-m\Delta u_{min}] + U_{max}G[-m\Delta u_{max}] + u \{G[-m\Delta u_{min}] - G[-m\Delta u_{max}]\}}{2}$$

where:

- m is a saturation factor that determines the smoothness of the transformation
- $\Delta u_{min} = u - U_{min}$
- $\Delta u_{max} = U_{max} - u$

After testing on multiple saturation values, we observed that a smaller m would cause a small overshoot ($\sigma \approx 0.116\%$ for $m = 5$) around the U_{min} and U_{max} values, but the saturation would be smoother; where a bigger saturation factor would cause no overshoot, but the transition between the linear and saturates phases would be more abrupt.

²This function has its medium value at 0, which is why it is necessary to add the $\frac{U_{min} + U_{max}}{2}$ in general, but we omitted the offset turn, because my limits are $U_{min} = -24$ V and $U_{max} = +24$ V, where the offset would be $\frac{U_{min} + U_{max}}{2} = 0$ V.

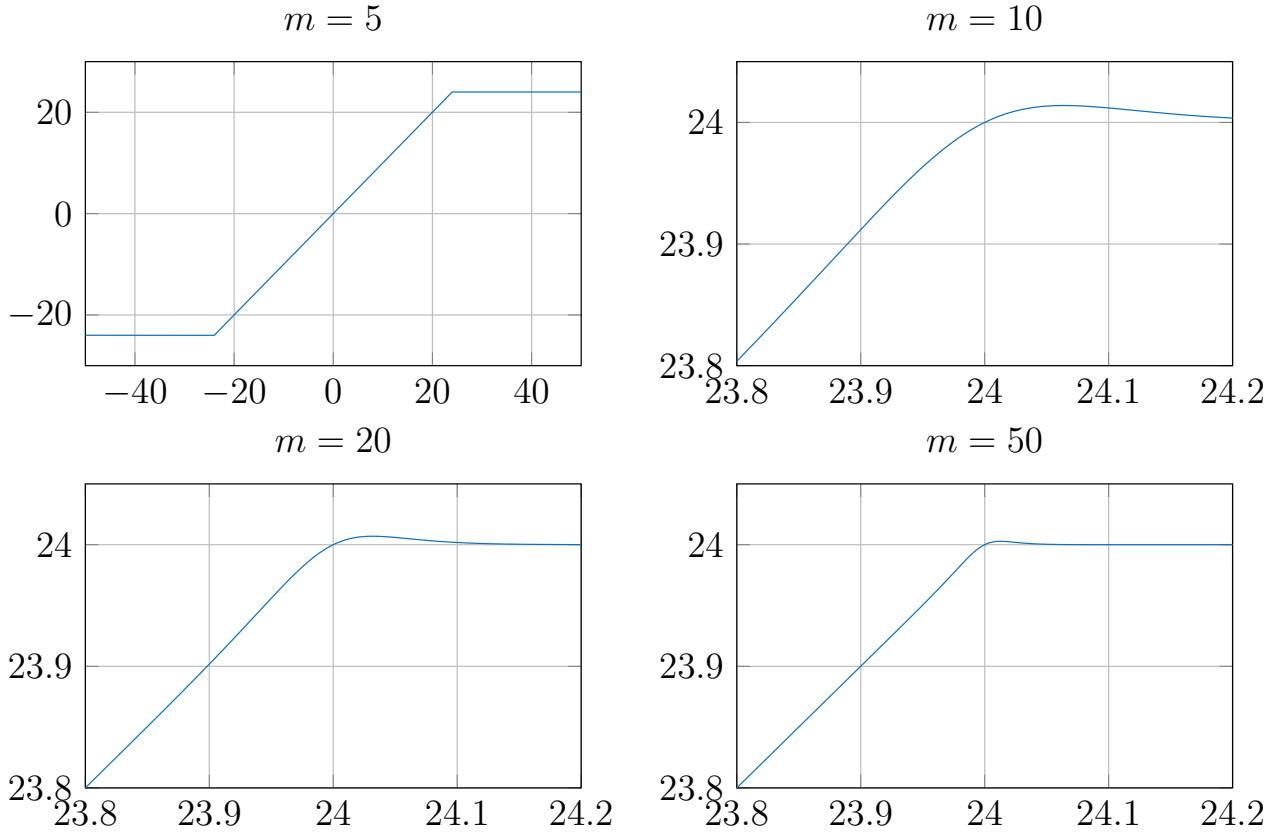


Figure 12: The graph of the $y(u)$ function for different m values, with the emphasis on the transition between the 2 states.

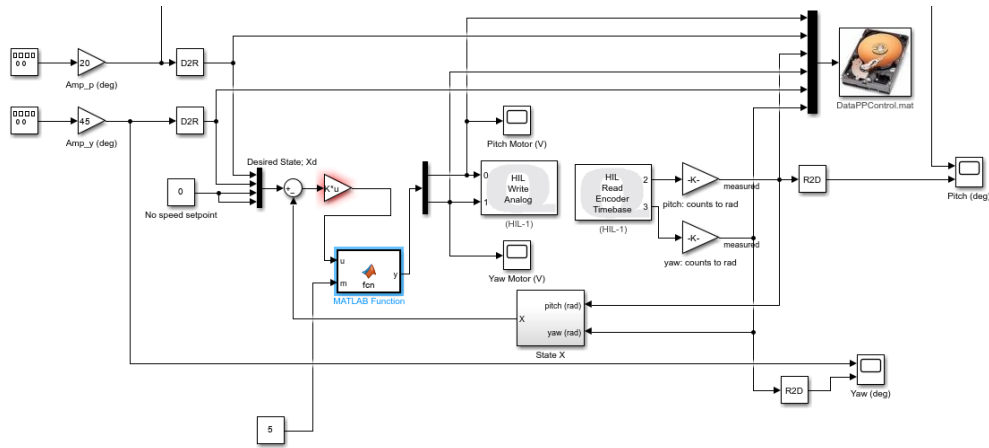


Figure 13: Simulink test scheme on the QUARC Simulator and on the real Quanser Aero 2 with the continuous approximation of the Saturation block

Because the overshoot was really low and because we prioritize the smoothness of the transition between the saturated state and the linear state, we implemented the `MATLAB Function` with $m = 5$, after which we ran all 3 simulations again to observe the changes in the command.

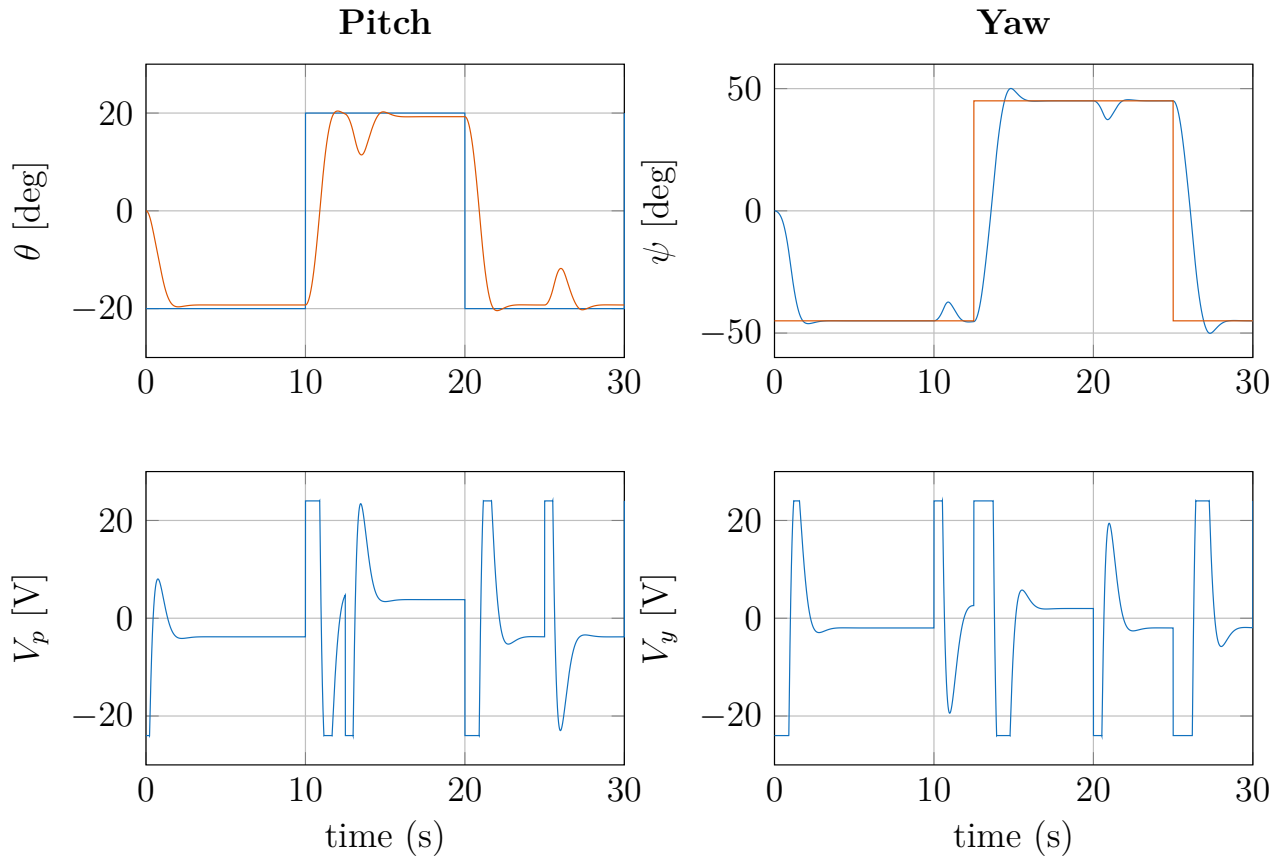


Figure 14: The response of the simulation process on the linear model with the tanh saturation

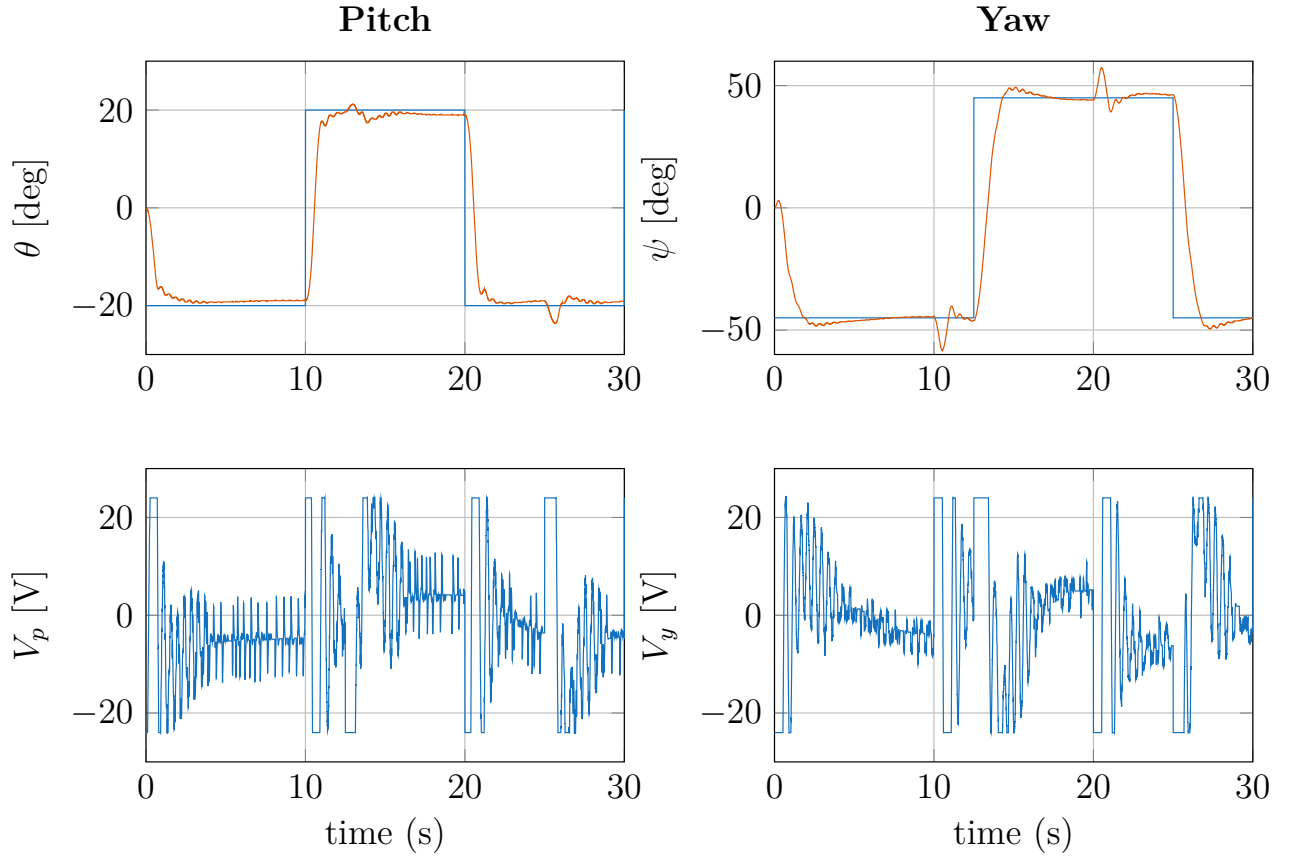


Figure 15: The response of the simulation process on the QUARC Simulator with the tanh saturation

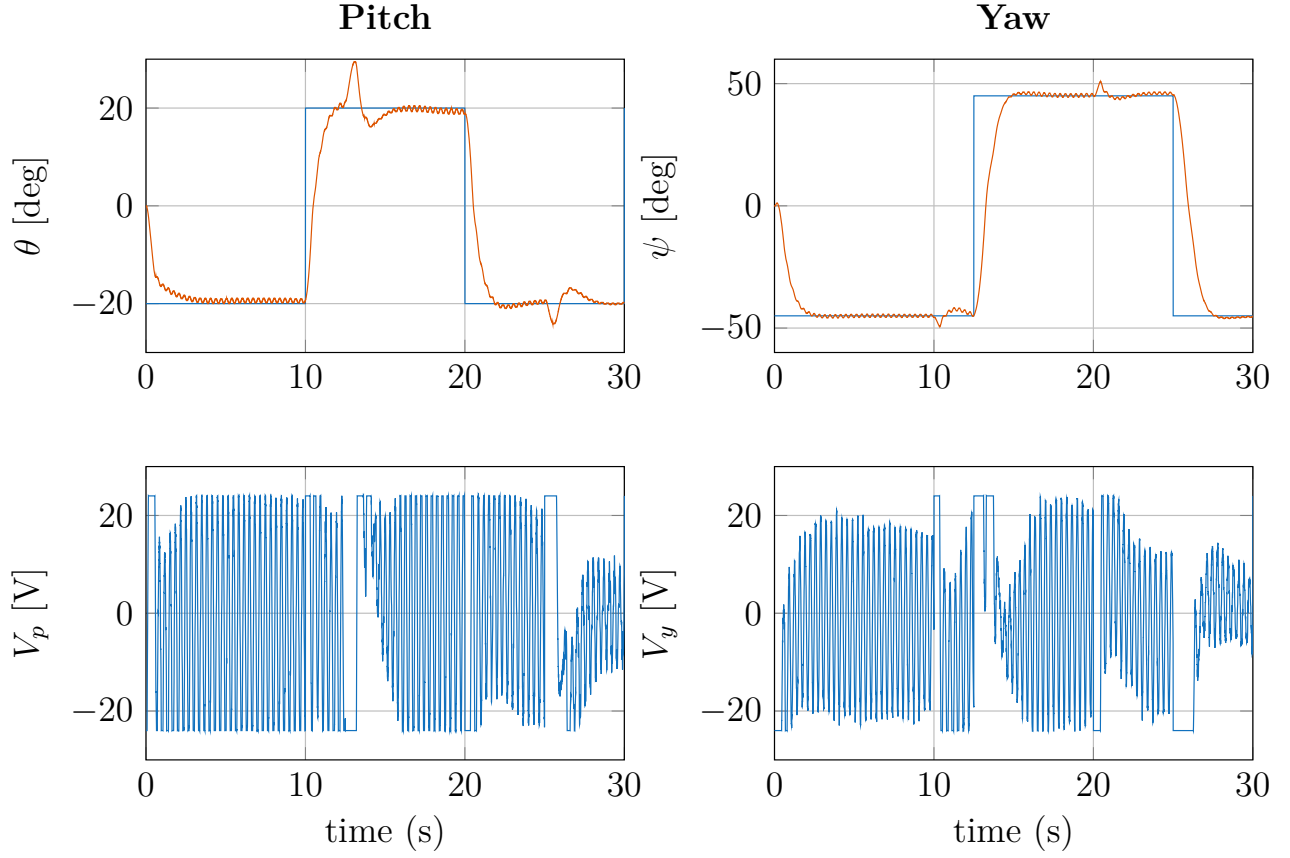


Figure 16: The response of the simulation process on the real system with the tanh saturation

As we can see, the changes are insignificant between the ideal saturation and the tanh saturation on the linear model and the QUARC simulator, but the command inputs given to the real system are smaller, while still being a square wave signal with high amplitude and high frequency, all while keeping the same performances.

In order to prevent the high frequency oscillations in the command, we added a **Rate Limiter** block in series with the tanh saturation on both the QUARC Simulator and the real plant and set the limits of the rate to ± 300 V/s, in order to have an impact on the input command without damaging the output performances.

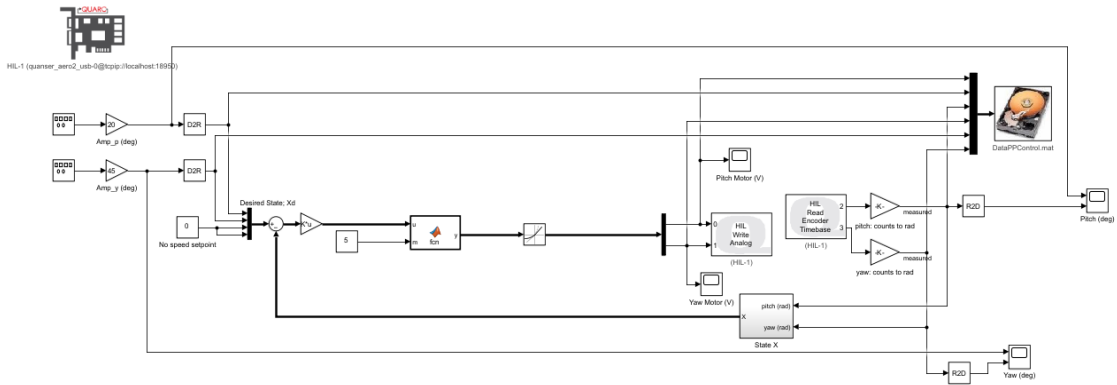


Figure 17: Simulink scheme with the rate limiter

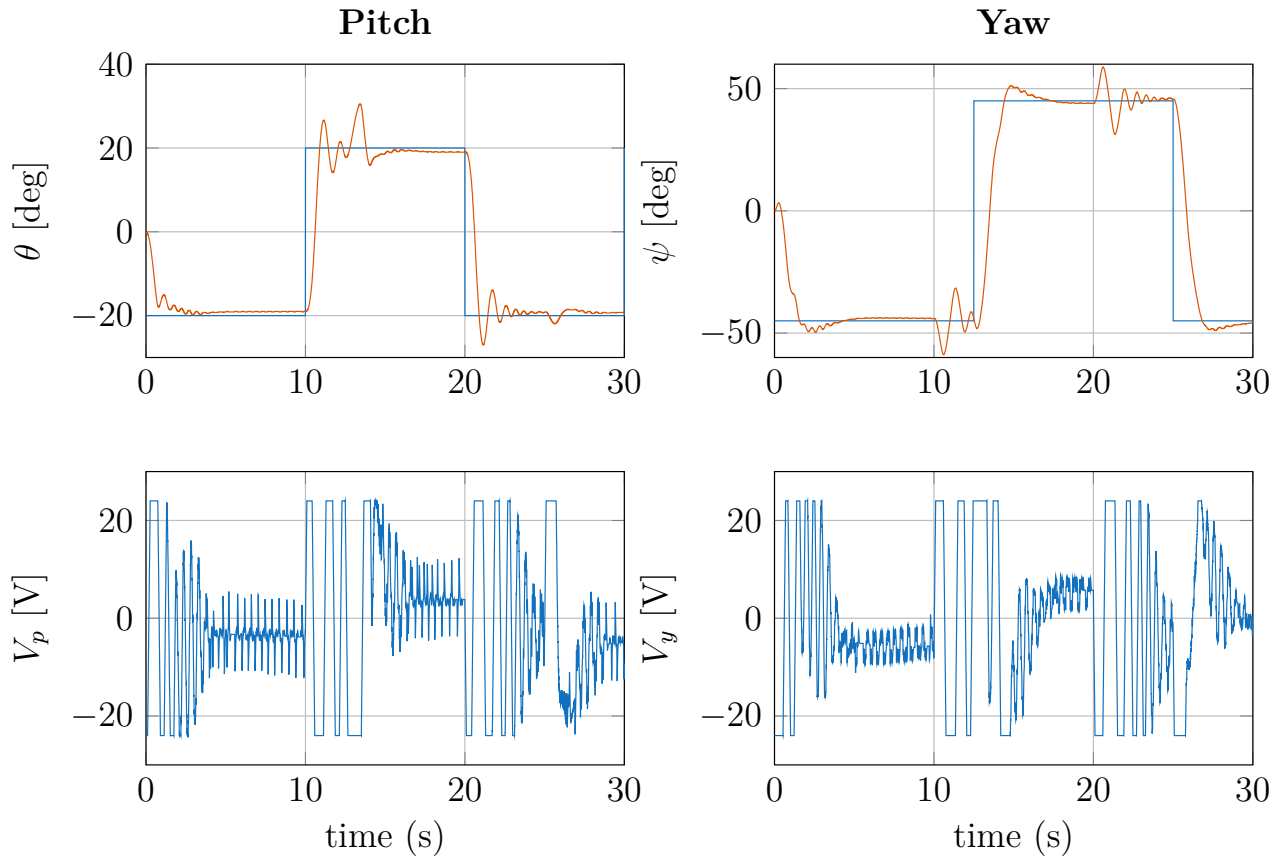


Figure 18: The response of the simulation process on the simulator with the tanh and rate saturation

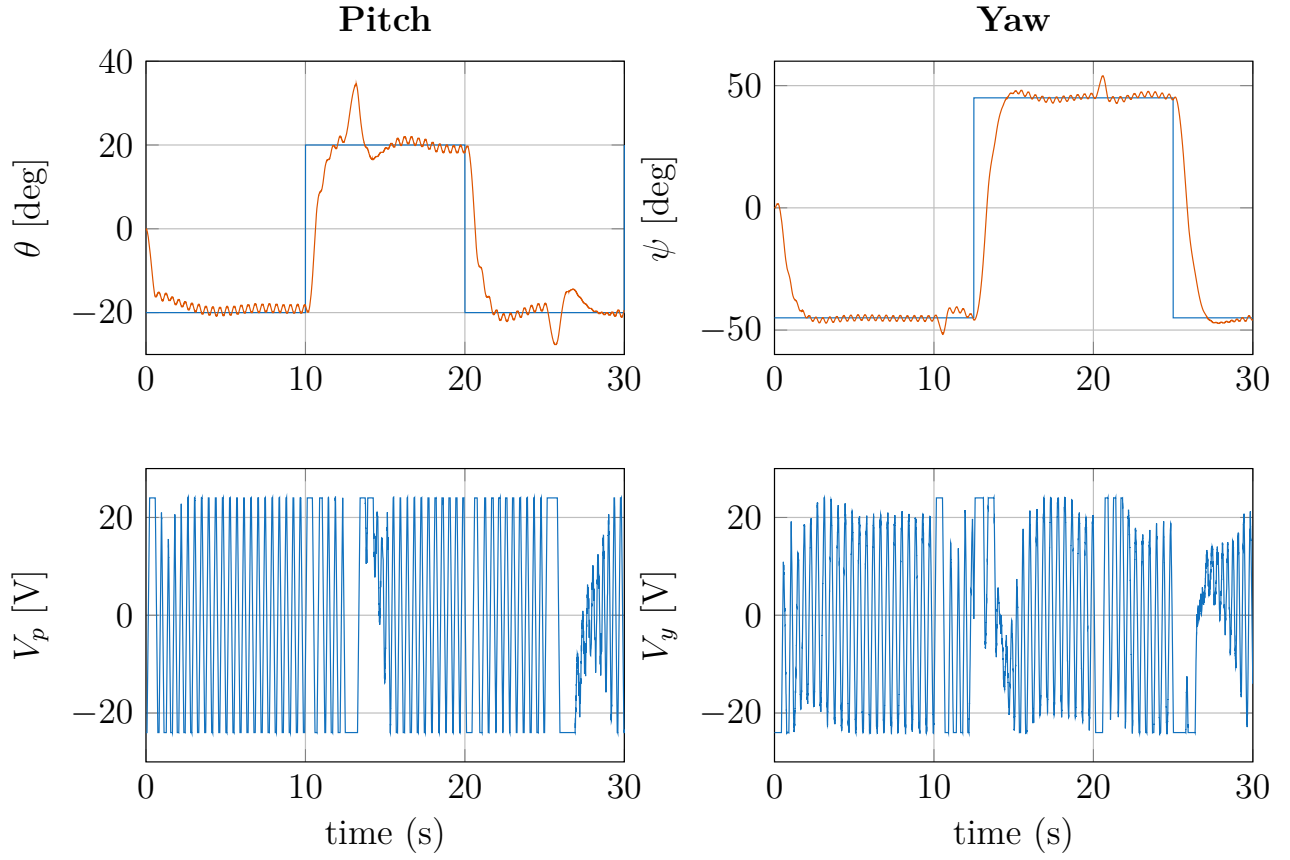


Figure 19: The response of the simulation process on the real plant with the tanh and rate saturation

The results show that the command still has high frequency oscillations, but now they are also present in the output. Because of this results, we decided to replace the rate limiter with a first-order low-pass filter with $K = 1$ and $\omega_c = 60$ rad/s, because it diminishes the high-frequency oscillations without totally canceling them.

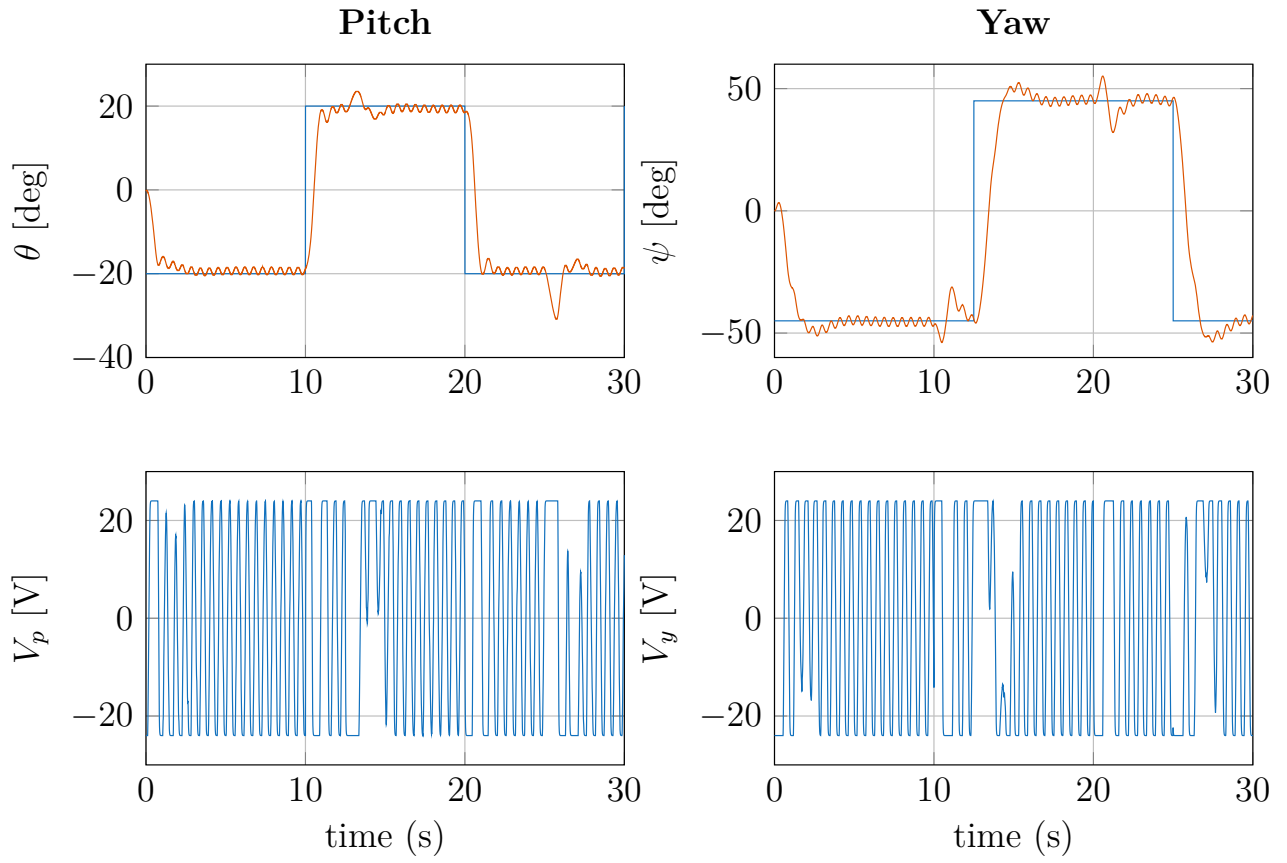


Figure 20: The response of the simulation process on the simulator with the tanh and the low-pass filter

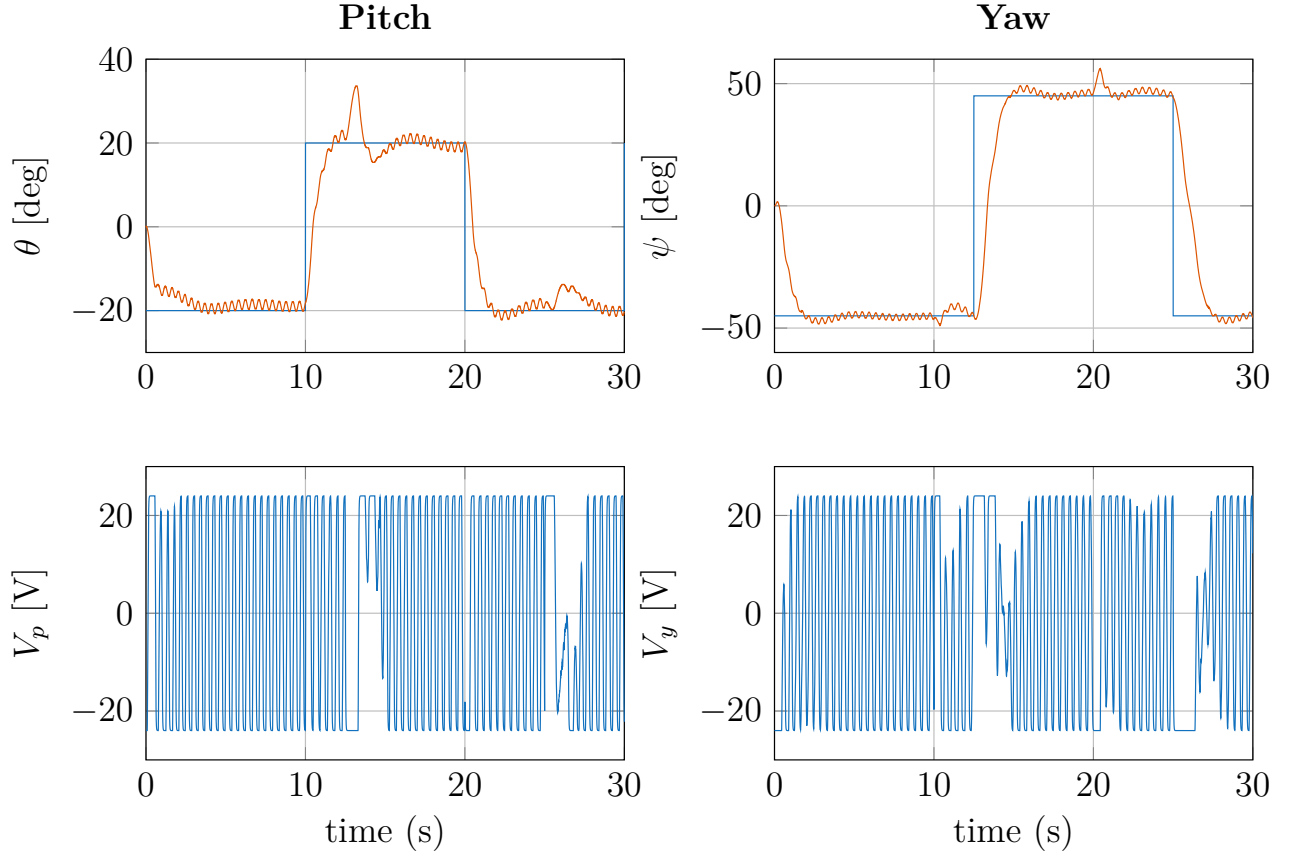


Figure 21: The response of the simulation process on the real plant with the tanh and low-pass filter

These results show that the low-pass filter and the rate saturation blocks have similar effects on the output, which has oscillations with small amplitudes in the steady state. If we increase the absolute value of the rate saturation limit or the cutoff frequency of the low-pass filter, it would increase the performances in the steady state, but it wouldn't smooth out the command inputs. Because of these effects, I will only keep the tanh saturation approximation.

The conclusion to the Pole Placement control is that it quite impossible to place the ideal poles to the ideal motions, as the `place` command doesn't guarantee the order of the pole placement on a MIMO system. On top of that, because the references are 2 square-wave signals with different frequencies, the reference change in one motion is seen as a perturbation in the other motion, these disturbances having a significantly higher impact on the system than the reference change of its own motion (e.g. the overshoot of the pitch motion when the yaw reference changed was much higher when the pitch reference changed its sign). Still, the command inputs were very high (especially on the real plant); which is why we need a control method that prioritizes the low energy of the command and the low energy of the state.

4.2 Linear Quadratic Regulator - MATLAB Implementation

The Linear Quadratic Regulator (LQR) is a control method implemented on MIMO systems that prioritizes the stability of the closed-loop system over an infinite-horizon with a minimal command and a minimal state, by setting our priorities through the matrices $Q \in \mathbb{R}^{n \times n}$, $Q = Q^T \geq 0$ and $R = R^T \in \mathbb{R}^{m \times m}$, $R > 0$. If $\|Q\| > \|R\|$, it means that we can tolerate a higher input magnitude if the state x is smaller and vice-versa.

The idea behind it is to solve this optimal problem with one constraint³:

$$\begin{aligned} \min_{u_n \in \mathbb{R}^m} J(x_0, u_n(t)) &= \min_{u_n \in \mathbb{R}^m} \int_0^{+\infty} (\|x(t)\|_{Q_c}^2 + \|u_n(t)\|_{R_c}^2) dt \\ \text{s.t.: } \dot{x}(t) &= Ax(t) + Bu_n(t) \end{aligned} \quad (10)$$

on a continuous system, respectively:

$$\begin{aligned} \min_{u_n \in \mathbb{R}^m} J(x_0, u_n[k]) &= \min_{u_n \in \mathbb{R}^m} \frac{1}{2} \sum_{k=0}^{+\infty} (\|x[k]\|_{Q_d}^2 + \|u_n[k]\|_{R_d}^2) \\ \text{s.t.: } x[k+1] &= A_d x[k] + B_d u_n[k] \end{aligned} \quad (11)$$

on a discrete system.

In order to obtain the stabilizing solution $u_n(t)$ on the continuous-time system, we will introduce $0 = x_0^T P_c x_0 - x_0^T P_c x_0$, where $P_c \in \mathbb{R}^{n \times n}$, $P_c = P_c^T \geq 0$ in the cost function:

$$J = x_0^T P_c x_0 - x_0^T P_c x_0 + \int_0^{+\infty} (x^T(t) Q_c x(t) + u_n^T(t) R_c u_n(t)) dt$$

where we notice that:

$$\int_0^{+\infty} \frac{d(x^T(t) P_c x(t))}{dt} dt = x^T(t) P_c x(t) \Big|_0^{+\infty} = x^T(\infty) P_c x(\infty) - x_0^T P_c x_0$$

Because the desired system must be stable, it means that $\lim_{t \rightarrow +\infty} x(t) = 0 \implies -x_0^T P_c x_0 = \int_0^{+\infty} \frac{d(x^T(t) P_c x(t))}{dt} dt$, so we introduce the $-x_0^T P_c x_0$ in the integrand, but not before computing the derivative $\frac{d(x^T(t) P_c x(t))}{dt}$ considering the state-space constraint from eq. (10):

$$\begin{aligned} \frac{d(x^T P_c x)}{dt} &= \dot{x}^T P_c x + x^T P_c \dot{x} \\ &= (x^T A^T + u_n^T B^T) P_c x + x^T P_c (Ax + Bu_n) \\ &= x^T (A^T P_c + P_c A) x + u_n^T B^T P_c x + x^T P_c B u_n \end{aligned}$$

Now we introduce the calculated derivative in the integrand:

$$J = x_0^T P_c x_0 + \int_0^{+\infty} [x^T (A^T P_c + P_c A + Q_c) x + u_n^T R_c u_n + u_n^T B^T P_c x + x^T P_c B u_n] dt$$

We would like to group the last 3 terms in the form $(u_n + Kx)^T R_c (u_n + Kx)$, so we would complete the square:

$$\begin{aligned} (u_n + Kx)^T R_c (u_n + Kx) &= u_n^T R_c u_n + u_n^T R_c Kx + x^T K^T R_c u_n + x^T K^T R_c Kx \\ &= u_n^T R_c u_n + u_n^T B^T P_c x + x^T P_c B u_n + x^T K^T R_c Kx \end{aligned}$$

from where we obtain

$$R_c K = B^T P_c \implies K = -R_c^{-1} B^T P_c$$

³we consider $\|x\|_M \triangleq x^T M x$ the 2-norm of the vector $x \in \mathbb{R}^n$ pondered with the matrix $M \in \mathbb{R}^{n \times n}$, $M = M^T \geq 0$

So, we introduce $0 = x^T P_c B R_c^{-1} B^T P_c - x^T P_c B R_c^{-1} B^T P_c$ in the integrand, so we obtain this form: where, in order to minimize it, we can make the integrand equal to 0 at any time, if:

$$\begin{cases} u_n(t) + R_c^{-1} B^T P_c x(t) = 0 \implies u_n(t) = -R_c^{-1} B^T P_c x(t) \\ A^T P_c + P_c A + Q_c - P_c B R_c^{-1} B^T P_c = O_n \end{cases}$$

The latter equation is known as the Continuous Algebraic Riccati Equation (CARE), which is guaranteed to have a unique stabilizing solution if:

- (A, B) is controllable
- $(Q_c^{\frac{1}{2}}, A)$ is observable, where $Q_c^{\frac{1}{2}}$ is the Cholesky factor

For the eq. (11), we will calculate the Lagrangian of the problem, where the cost function is $\frac{1}{2} \sum_{k=0}^{\infty} (x^T[k] Q_d x^T[k] + u_n^T[k] R_d u_n[k])$ and the equality constraint determined by the state equation of the system ($x[k+1] = A_d x[k] + B_d u_n[k]$), to which we attribute the co-state $\lambda[k] \in \mathbb{R}^n$. The constrained part of the Lagrangian ends up being $f(x, u_n, \lambda) \triangleq \lambda^T[k+1] (A_d x[k] + B_d u_n[k] - x[k+1])$, but we will split it into $f = \frac{1}{2}(f + f^T)$ for consistency with the cost function, despite the fact it's a numerical function: $f(x, u_n, \lambda), L(x, u_n, \lambda) \in \mathbb{R}$

$$\begin{aligned} L(x, u_n, \lambda) &= \frac{1}{2} \sum_{k=0}^{\infty} (x^T[k] Q_d x^T[k] + u_n^T[k] R_d u_n[k]) \\ &\quad + \frac{1}{2} \sum_{k=0}^{\infty} \lambda^T[k+1] (A_d x[k] + B_d u_n[k] - x[k+1]) \\ &\quad + \frac{1}{2} \sum_{k=0}^{\infty} (A_d x[k] + B_d u_n[k] - x[k+1])^T \lambda[k+1] \end{aligned}$$

for which we have to calculate its critical points:

$$\begin{cases} \frac{\partial L}{\partial x[k]} = 0 & \implies Q_d x[k] + A_d^T \lambda[k+1] - \lambda[k] = 0 \implies \lambda[k] = Q_d x[k] + A_d^T \lambda[k+1] \\ \frac{\partial L}{\partial u_n[k]} = 0 & \implies R_d u_n[k] + B_d^T \lambda[k+1] = 0 \implies u_n[k] = -R_d^{-1} B_d^T \lambda[k+1] \\ \frac{\partial L}{\partial \lambda[k]} = 0 & \implies A_d x[k] + B_d u_n[k] = x[k+1] \end{cases}$$

And we obtain this system of equations between the state $x \in \mathbb{R}^n$ and the co-state $\lambda \in \mathbb{R}^n$ called the Hamiltonian system:

$$\begin{aligned} \lambda[k] &= Q_d x[k] + A_d^T \lambda[k+1] \\ x[k+1] &= A_d x[k] - B_d R_d^{-1} B_d^T \lambda[k+1] \end{aligned} \tag{12}$$

We assume that the co-state $\lambda[k]$ could be obtained as $\lambda[k] = P_d[k] x[k]$, where $P[k] = P^T[k] \geq 0$ and $\lim_{k \rightarrow +\infty} P_d[k] = O_n$. We substitute the co-state in the Hamiltonian system:

$$\begin{aligned} P_d[k] x[k] &= Q_d x[k] + A_d^T P_d[k+1] x[k+1] \\ x[k+1] &= A_d x[k] - B_d R_d^{-1} B_d^T P_d[k+1] x[k+1] \end{aligned} \tag{13}$$

Which can be written in a matrix-vector form:

$$\begin{bmatrix} x[k+1] \\ \lambda[k] \end{bmatrix} = \begin{bmatrix} A_d & -B_d R_d^{-1} B_d^T \\ Q_d & A_d^T \end{bmatrix} \begin{bmatrix} x[k] \\ \lambda[k+1] \end{bmatrix}$$

We observed that we got a system where the time axis for the co-state goes backwards (we use $\lambda[k+1]$ in order to get $\lambda[k]$).

From the second equation of the Hamiltonian system eq. (13), we calculate $x[k+1]$ by $x[k]$, where we obtain:

$$(I_n + B_d R_d^{-1} B_d^T P_d[k+1]) x[k+1] = A_d x[k]$$

We will calculate the determinant after the Schur complement: $\det(I_n + XY) = \det(I_m + YX)$, $X \in \mathbb{R}^{n \times m}$, $Y \in \mathbb{R}^{m \times n}$:

$$\begin{aligned} \det(I_n + B_d R_d^{-1} B_d^T P_d[k+1]) &= \det(I_m + B_d^T P_d[k+1] B_d R_d^{-1}) \\ &= \det(R_d R_d^{-1} + B_d^T P_d[k+1] B_d R_d^{-1}) \\ &= \det(R_d + B_d^T P_d[k+1] B_d) \det(R_d^{-1}) \end{aligned}$$

We know that

$$\begin{cases} R_d > 0 \implies R_d^{-1} > 0 \implies \det(R_d^{-1}) > 0 \\ R_d > 0, P_d[k+1] \geq 0 \implies \det(R_d + B_d^T P_d[k+1] B_d) > 0 \end{cases}$$

which means that $\det(I_n + B_d R_d^{-1} B_d^T P_d[k+1]) \neq 0$, so we can calculate $(I_n + B_d R_d^{-1} B_d^T P_d[k+1])^{-1}$, according to the Woodbury inversion lemma from [3] ($X = I_n$, $U = B_d$, $C = R_d^{-1}$, $V = B_d^T P_d[k+1]$):

$$(X + UCV)^{-1} = X^{-1} - X^{-1}U(C^{-1} + VX^{-1}U)^{-1}VX^{-1}$$

$$(I_n + B_d R_d^{-1} B_d^T P_d[k+1])^{-1} = I_n - B_d (R_d + B_d^T P_d[k+1] B_d)^{-1} B_d^T P_d[k+1]$$

So we get this formula between two consecutive states:

$$x[k+1] = \left[A_d - B_d (R_d + B_d^T P_d[k+1] B_d)^{-1} B_d^T P_d[k+1] A_d \right] x[k]$$

with which we substitute $x[k+1]$ in the first equation of the Hamiltonian system:

$$\begin{aligned} P_d[k]x[k] &= Q_d x[k] + A_d^T P_d[k+1] \left[A_d - B_d (R_d + B_d^T P_d[k+1] B_d)^{-1} B_d^T P_d[k+1] A_d \right] x[k] \\ &= \left[Q_d + A_d^T P_d[k+1] A_d - A_d^T P_d[k+1] B_d (R_d + B_d^T P_d[k+1] B_d)^{-1} B_d^T P_d[k+1] A_d \right] x[k] \end{aligned}$$

If we move everything to the left side and divide by $x[k]$, we get:

$$P_d[k] - Q_d - A_d^T P_d[k+1] A_d + A_d^T P_d[k+1] B_d (R_d + B_d^T P_d[k+1] B_d)^{-1} B_d^T P_d[k+1] A_d = O_n$$

we reverse the time base for the co-state ($k \leftrightarrow k+1$), so $P[0] = O_n$ and $\lim_{k \rightarrow \infty} P_d[k] = P_d = P_d^T \geq 0$. Now, as $k \rightarrow \infty$, the equation becomes:

$$P_d - Q_d - A_d^T P_d A_d + A_d^T P_d B_d (R_d + B_d^T P_d B_d)^{-1} B_d^T P_d A_d = O_n \quad (14)$$

called the Discrete Algebraic Riccati Equation (DARE), which is guaranteed to have a stabilizing solution P_d under the same conditions as the CARE.

We tested the LQR regulator similarly as the Pole Placement regulator, by starting from this scheme with the same approximation of the saturation function:

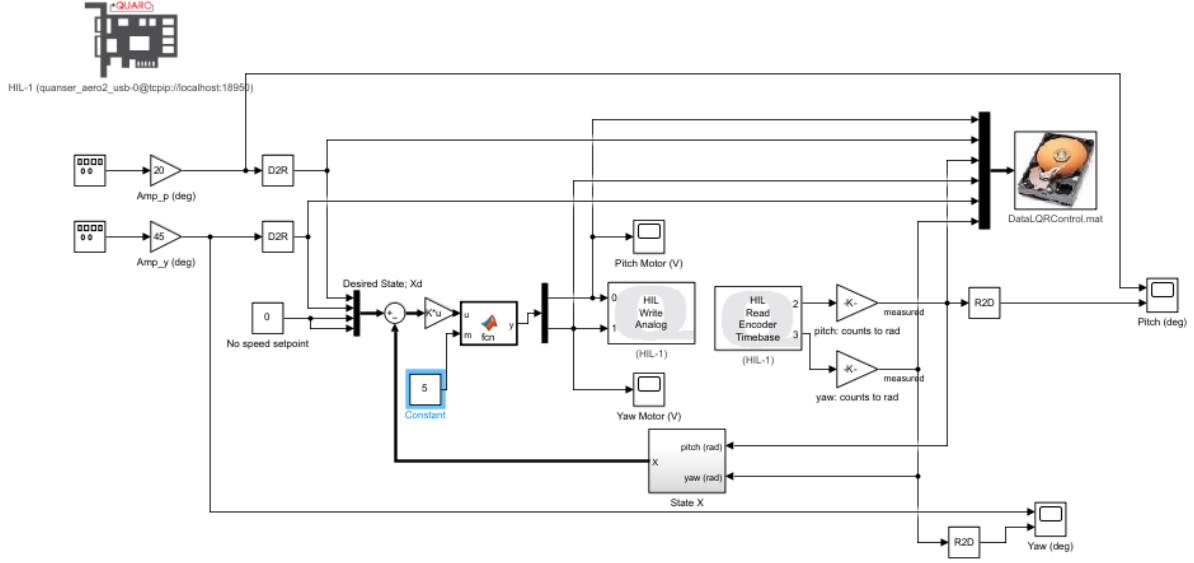


Figure 22: Simulink scheme used for the LQR implementation on the QUARC Simulator and the real plant

These schemes actually try to minimize both the command, and, instead of the state x of the system, they try to minimize the difference between the real state x and the reference x^{ref} , composed of the 2 square-wave signals for the positions used in the Pole-Placement simulation and 0 for the angular speeds with the K gain calculated by solving the classical LQR problem.

For this implementation, we used these matrices:

$$Q = \begin{bmatrix} 10,000 & 0 & 0 & 0 \\ 0 & 1,000 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 50 \end{bmatrix}$$

$$R = 0.1I_2$$

The reason for choosing diagonal matrices is because we could choose how much we would like to minimize the error on each state and command input. If we take the state component from the optimal gain with this Q matrix, we obtain these sum in the cost function where we minimize the error between the output state and the reference state $\Delta x = x_{ref} - x$:

$$[\Delta\theta \quad \Delta\psi \quad \Delta\dot{\theta} \quad \Delta\dot{\psi}] \begin{bmatrix} 10,000 & 0 & 0 & 0 \\ 0 & 1,000 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 50 \end{bmatrix} \begin{bmatrix} \Delta\theta \\ \Delta\psi \\ \Delta\dot{\theta} \\ \Delta\dot{\psi} \end{bmatrix} = 10,000 \cdot \Delta\theta^2 + 1,000 \cdot \Delta\psi^2 + 10 \cdot \Delta\dot{\theta}^2 + 50 \cdot \Delta\dot{\psi}^2$$

Because we prioritize the tracking of the position reference, we chose much higher coefficients in the Q matrix for the angular positions than for the angular speeds. Also, because we already have an integrator in the transfer functions for the yaw motion, but there is none in the pitch motion, we chose the coefficient for the pitch motion to be 10 times bigger than the one for the yaw motion.

After computing the `lqr` command, we obtained the gain

$$K = \begin{bmatrix} 275.7010 & -42.1363 & 136.8680 & -27.1453 \\ 127.4371 & 90.6892 & 66.9148 & 60.6914 \end{bmatrix}$$

which stabilizes the closed-loop system in these poles:

$$\Lambda = \{-1.9635 \pm 2.0325i, -1.6429 \pm 1.4560i\}$$

all of them being stable poles.

On the linear model, we obtained these results:

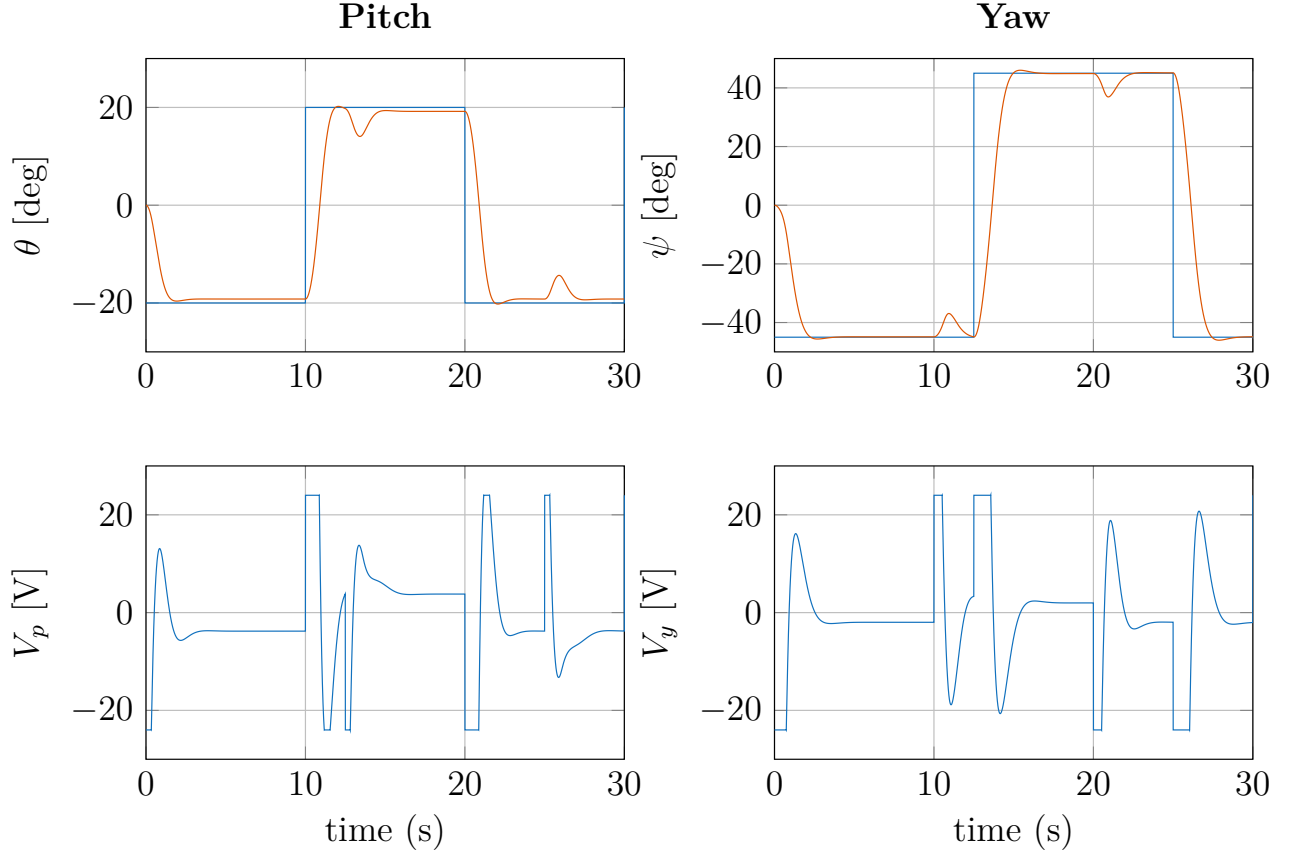


Figure 23: Results of the LQR simulation on the linear model

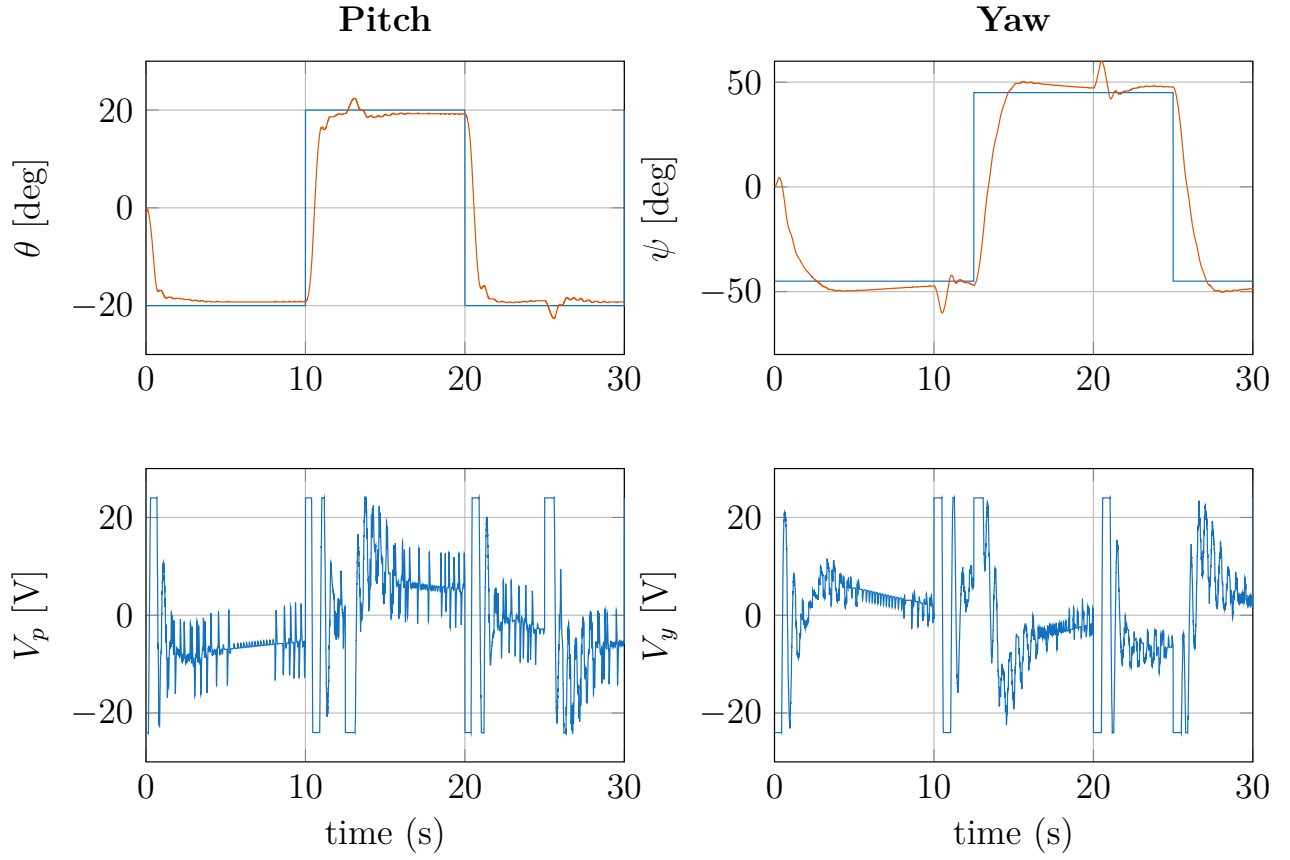


Figure 24: Results of the LQR simulation on the QUARC Simulator

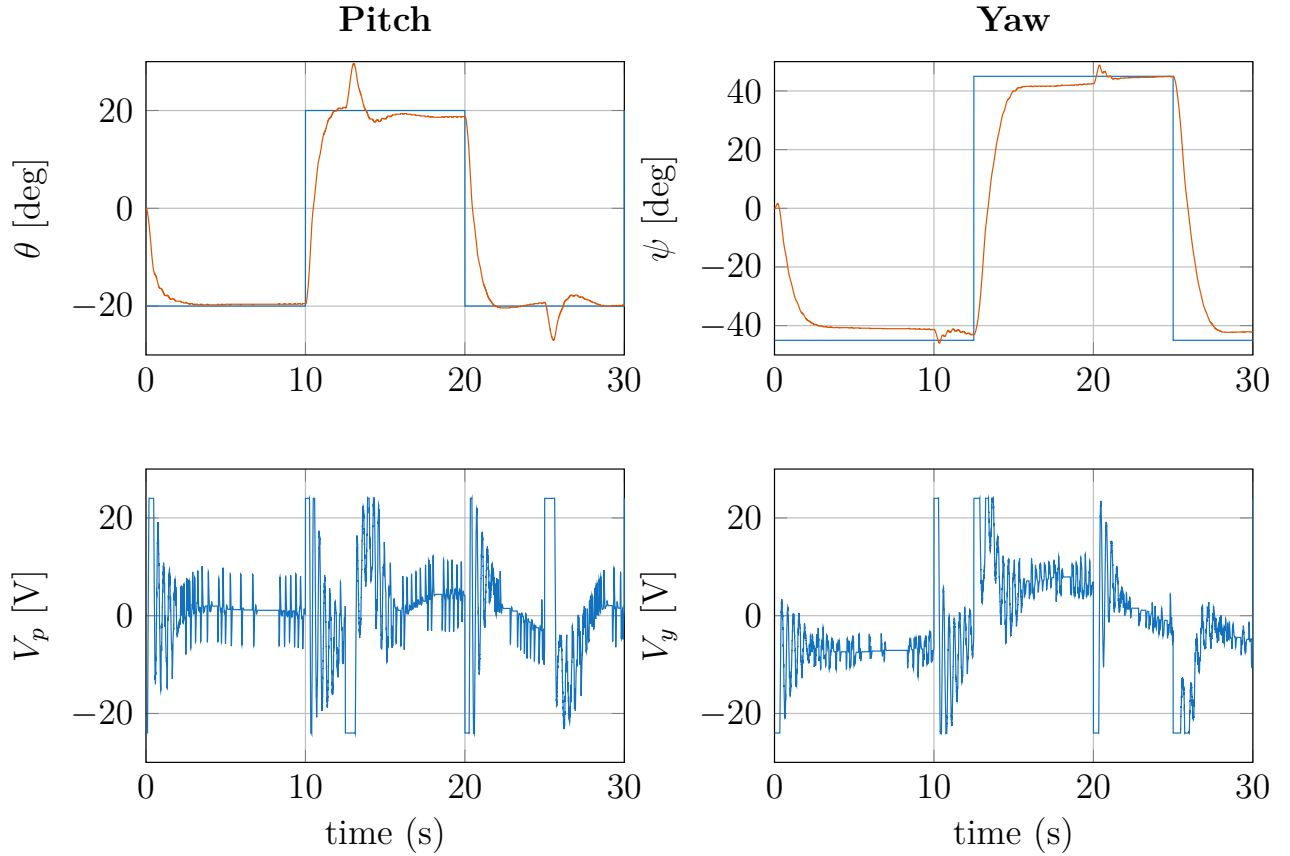


Figure 25: Results of the LQR simulation on the real plant

Pitch Axis (θ)			
Parameter	Linear Model	Simulator	Real Plant
Relative steady state error (ϵ)	$\approx 5.2\%$	$\approx 5.6\%$	$\approx 3.1\%$
Overshoot (σ)	$\approx 5.3\%$	$\approx 2.64\%$	$\approx 7.74\%$
Peak time (t_p)	2 s	2.7 s	2.4 s
Coupling overshoot (σ')	$\approx -25.21\%$	$\approx +13.24\%$	+40%

Yaw Axis (ψ)			
Parameter	Linear Model	Simulator	Real Plant
Relative steady state error (ϵ)	$\approx 0.3\%$	4.5%	0
Overshoot (σ)	$\approx 2.54\%$	$\approx 11.33\%$	0
Peak time (t_p)	2.7 s	2.25 s	-
Coupling overshoot (σ')	$\approx -17.71\%$	$\approx +34.4\%$	$\approx 12.7\%$

Table 2: Performance comparison of the LQR Controller across the three tests

Based on the results from the 3 graphics and the table, the LQR controller assured command inputs with lower magnitudes (compared to the PP controller), but the overall performances are comparable:

- the steady-state errors are smaller compared to the PP controller
- the overshoots are bigger than the ones in the PP controller
- the closed-loop system is more sensitive to disturbances, as emphasized by the bigger coupling overshoots(+)/undershoots(-)

The LQR method assured lower command inputs compared to the PP controller, but it's more sensitive to the reference changes and it doesn't assure the tracking of the references, which is why we have to include an integrator.

4.3 Linear Quadratic Integrator - MATLAB Implementation

In order to prioritize the 0 steady-state error, we have to augment the state x with $\int \theta$ and $\int \psi$:

$$x_a = \begin{bmatrix} \int \theta(t)dt \\ \int \psi(t)dt \\ \theta \\ \psi \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$$

which leads to an augmented state-space representation, even if we keep the same command inputs u :

$$\begin{aligned} \dot{x}_a &= A_a x_a + B_a u \\ y &= C_a x_a \end{aligned} \tag{15}$$

where the matrices are:

$$A_a = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & -\frac{K_{sp}}{J_p} & 0 & -\frac{D_p}{J_p} & 0 \\ 0 & 0 & 0 & 0 & 0 & -\frac{D_y}{J_y} \end{bmatrix}$$

$$B_a = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ \frac{D_t K_{pp}}{J_p} & \frac{D_t K_{py}}{J_p} \\ \frac{D_t K_{yp}}{J_y} & \frac{D_t K_{yy}}{J_y} \end{bmatrix}$$

$$C_a = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Before we would implement the LQR algorithm on this augmented model, I will test if it's controllable by computing the $\mathcal{R}_a = [B \ AB \ A^2B \ A^3B \ A^4B \ A^5B]$ matrix, whose rank is equal to $6 = n + 2$, which means the system (A_a, B_a) is controllable.

The purpose of this method is to minimize the error between the reference functionality defined by x_a^{ref} and u_{ref} , and the actual functioning, defined by $x_a(t)$ and $u(t)$, the cost function being:

$$J_{LQI} = \int_0^{+\infty} (\|x_a - x_a^{ref}\|_{Q_a} + \|u - u_{ref}\|_{R_a}) dt$$

In our case, when the reference is described by constant values, we can calculate online the u_{ref} by solving this linear system:

$$\dot{x}_{ref} = A_a x_a^{ref} + B_a u_{ref} \implies B_a \cdot u_{ref} = \dot{x}_{ref} - A_a x_a^{ref}$$

which we encapsulate in a MATLAB Subsystem where we calculate the

Because of this cost function, we compute the state-feedback K with the function `lqr(Aa, Ba, Qa, Ra)`, where we obtain:

$$u^* - u_{ref} = -K (x_a - x_a^{ref}) \implies u^* = u_{ref} - K (x_a - x_a^{ref})$$

The Simulink models start from the LQR models with the addition of the `u_ref` function, as well as adding the integrator blocks to the angular position in order to append these to the initial state:

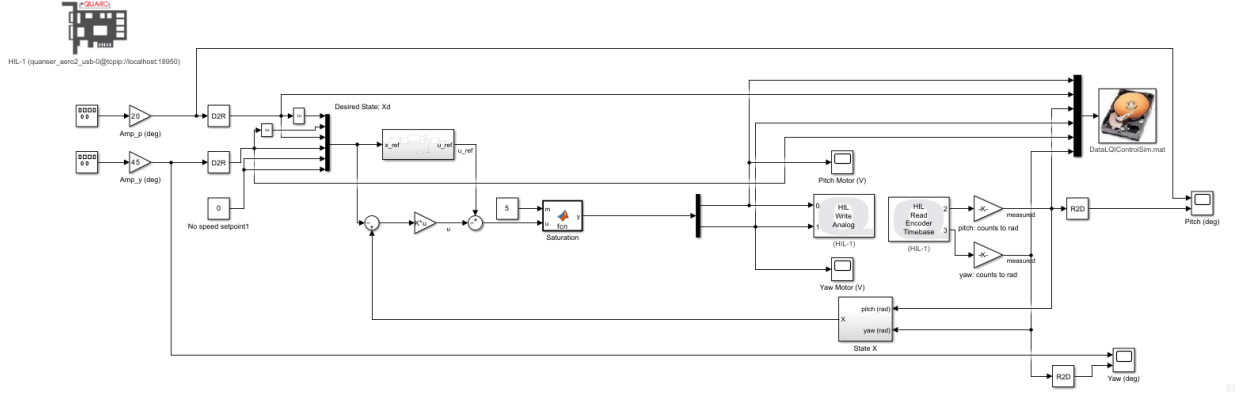


Figure 26: Simulink model for the LQI implementation on the QUARC Simulator and the real plant

Similarly to the LQR controller, we chose diagonal matrices Q_a and R_a to control the impact of each input voltage and state separately, with high coefficients for the position and angular speeds to follow the reference positions and minimize the effects of changing the reference on the other axis:

$$Q_a = \begin{bmatrix} 10^{-1} & 0 & 0 & 0 & 0 & 0 \\ 0 & 10^{-2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 50,000 & 0 & 0 & 0 \\ 0 & 0 & 0 & 50,000 & 0 & 0 \\ 0 & 0 & 0 & 0 & 20,000 & 0 \\ 0 & 0 & 0 & 0 & 0 & 10,000 \end{bmatrix}$$

$$R_a = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2$$

By computing the $\text{lqr}(A_a, B_a, Q_a, R_a)$, we obtain the gain matrix:

$$K = \begin{bmatrix} 0.2845 & -0.0437 & 190.4127 & -97.6735 & 169.0129 & -58.6637 \\ 0.1381 & 0.0900 & 92.0078 & 201.2154 & 83.5113 & 123.9346 \end{bmatrix}$$

which stabilizes the augmented system in the close-loop poles:

$$\Lambda_a = \left\{ \begin{array}{l} -4.063439071834646, \\ -1.979276962041696, \\ -2.748607032263342 + 0.132457337257551i, \\ -2.748607032263342 - 0.132457337257551i, \\ -0.001411913044831, \\ -0.000447213604447 \end{array} \right\}$$

An important observation is that the closed-loop system is stable, but it's not robust, as the poles associated with the angular speeds are really close to the imaginary axis.

We simulated the behavior of the linear augmented system, the QUARC Simulator and the real plant with the same references for the angular positions.

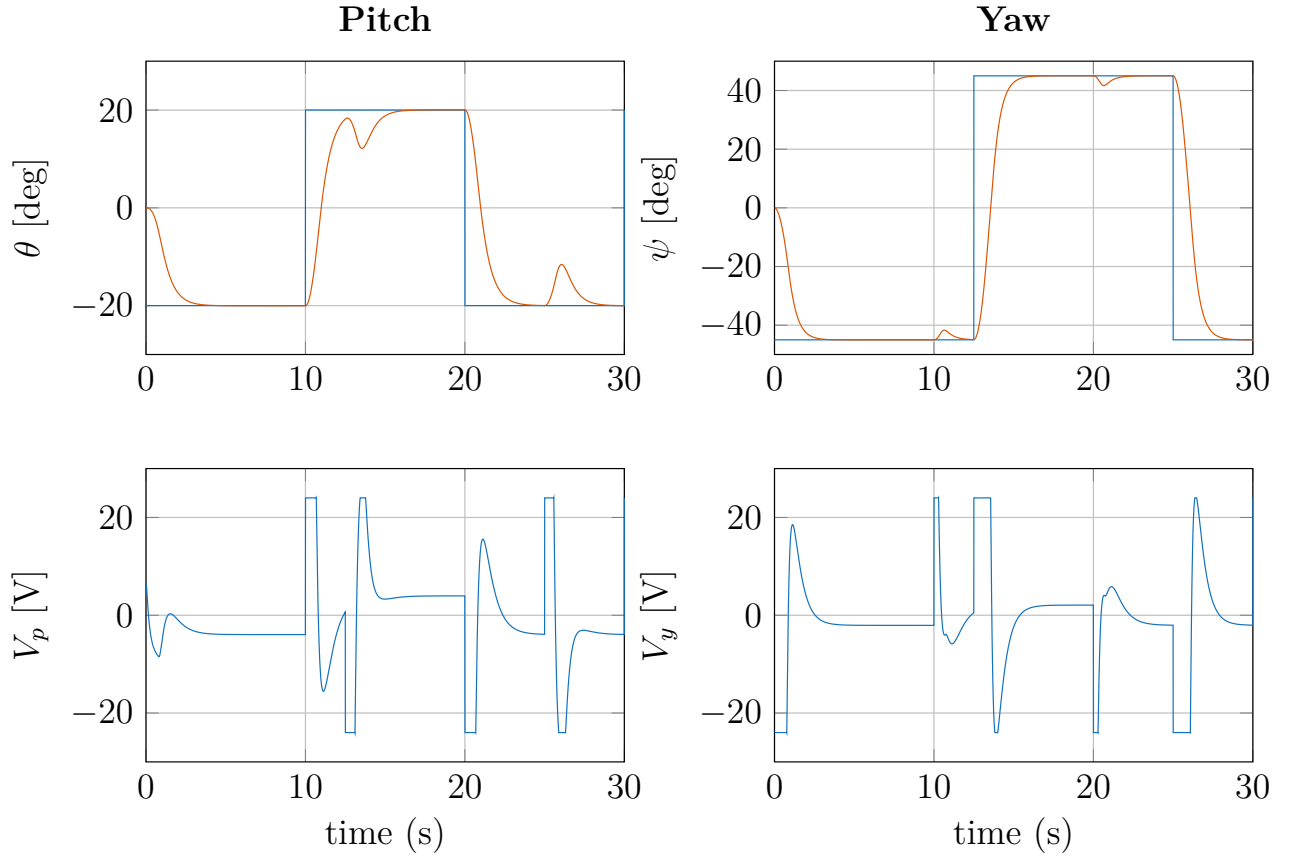


Figure 27: Results of the LQI simulation on the linear model

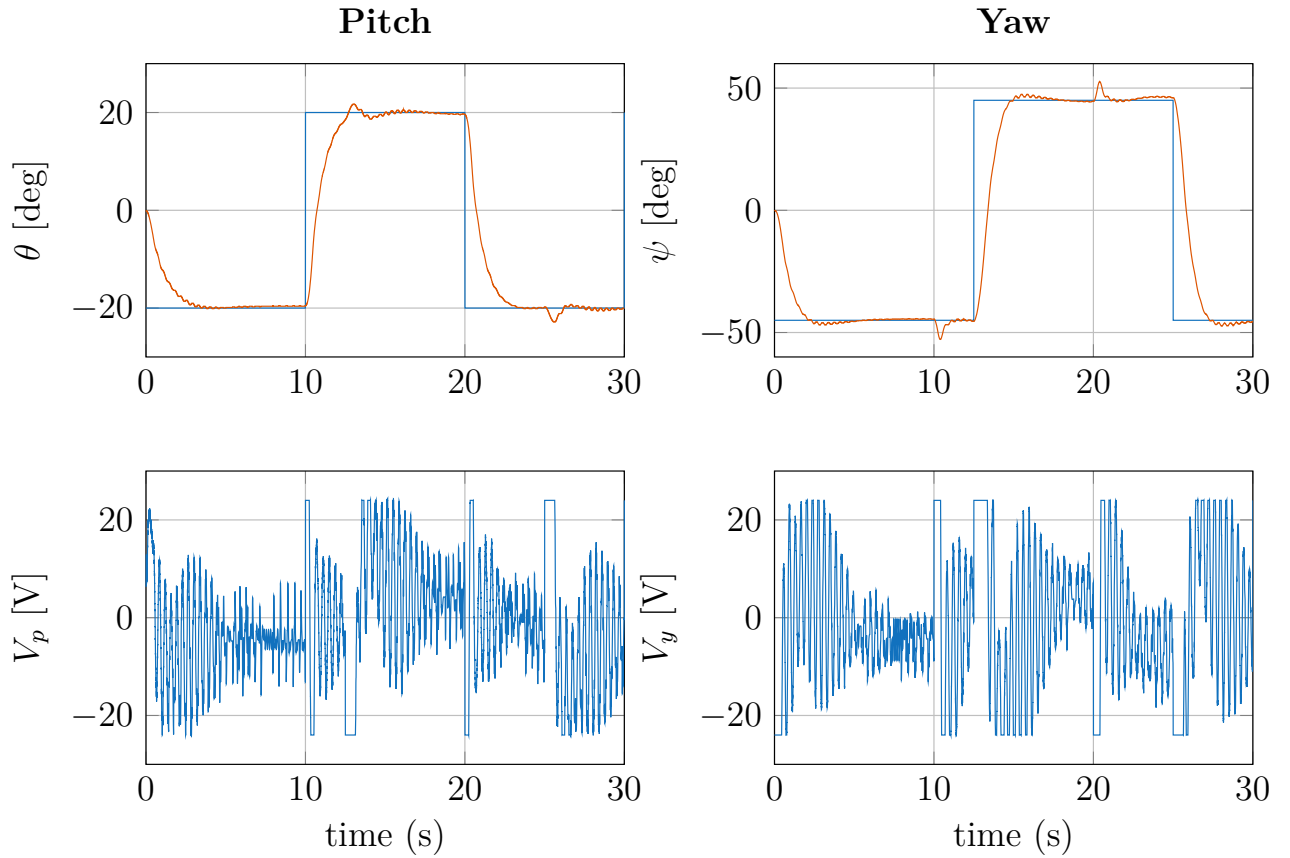


Figure 28: Results of the LQI simulation on the QUARC Simulator

The pitch motion has:

- no overshoot
- no steady-state error
- big settling time at any reference change

The yaw motion has:

- no steady-state error
- small overshoot $\sigma \approx 5.5\%$
- big settling time

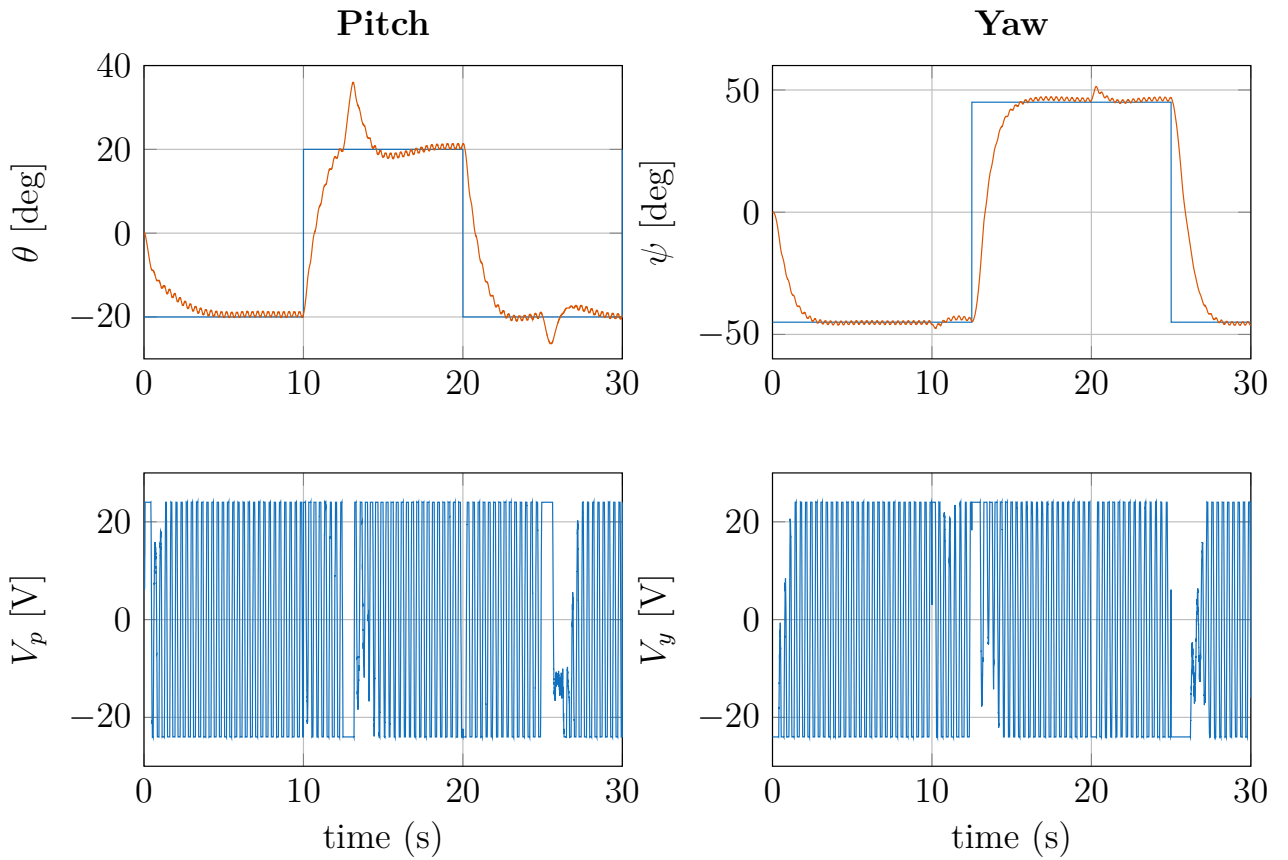


Figure 29: Results of the LQI simulation on the real plant

The real plant reaches similar performances with the QUARC Simulator, but, because of the results from testing the models, the input magnitudes are much higher, being comparable to the ones generated by the PP controller, while the pitch axis was much more sensitive to the changes in the yaw motion than the simulator, as the coupling overshoot was bigger than 50%.

Because of these results, we decided to adjust the Q_a and R_a matrices in order to diminish the input magnitudes, as well as reducing the disturbances in the pitch motion caused by the

change in the yaw motion, ending up with these matrices:

$$Q_a = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 10^{-1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 50,000 & 0 & 0 & 0 \\ 0 & 0 & 0 & 50,000 & 0 & 0 \\ 0 & 0 & 0 & 0 & 100,000 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1,000 \end{bmatrix}$$

$$R_a = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = 2I_2$$

which leads to the gain matrix:

$$K = \begin{bmatrix} 0.6407 & -0.0946 & 133.4127 & -66.9437 & 221.5603 & -33.8632 \\ 0.2992 & 0.2026 & 61.6658 & 143.3740 & 104.5246 & 75.0501 \end{bmatrix} \quad (16)$$

which stabilizes the system in the closed-loop poles:

$$\Lambda = \left\{ \begin{array}{l} -5.5309, \\ -2.0274 \pm 1.8760i, \\ -0.7228, \\ -0.0045, \\ -0.0014 \end{array} \right\}$$

The performances on the yaw axis are really good with a smaller command input, despite the small oscillations around the setpoint.

On the pitch axis, the disturbances caused by the shift of the yaw reference are diminished, but the settling time is bigger. On top of it, the magnitude of the V_p command input is still as high as the one tested first.

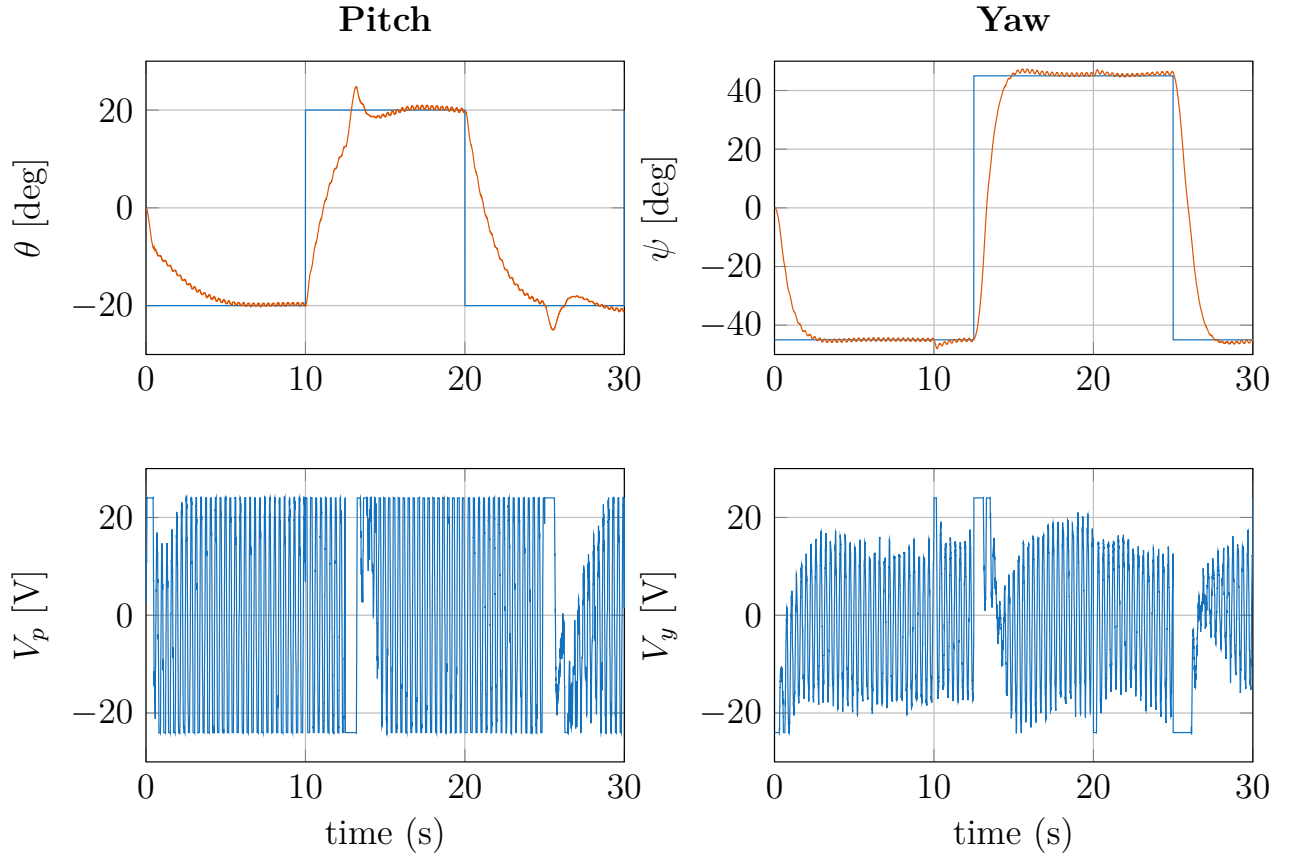


Figure 30: Results of the LQI simulation on the real plant on a second attempt

We tried to decrease this command input by increasing the $R_{a_{11}}$ term from 2 to 5, while keeping both $R_{a_{22}}$ and $R_{a_{33}}$ at 2, while it improves greatly the input on both command inputs, it deteriorates the performances on the pitch axis, as it will have a steady-state error.

By using the $R_a = \begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$ instead of $2I_2$, we obtain the gain matrix:

$$K = \begin{bmatrix} 0.4025 & -0.0616 & 79.9848 & -43.6072 & 145.0384 & -26.1824 \\ 0.3081 & 0.2013 & 63.9365 & 142.4263 & 106.5008 & 75.3098 \end{bmatrix}$$

which stabilizes the system in the closed-loop poles:

$$\Lambda = \left\{ \begin{array}{l} -3.7788, \\ -1.9357 \pm 1.7743i, \\ -0.7432, \\ -0.0044, \\ -0.0014 \end{array} \right\}$$

and these are the results:

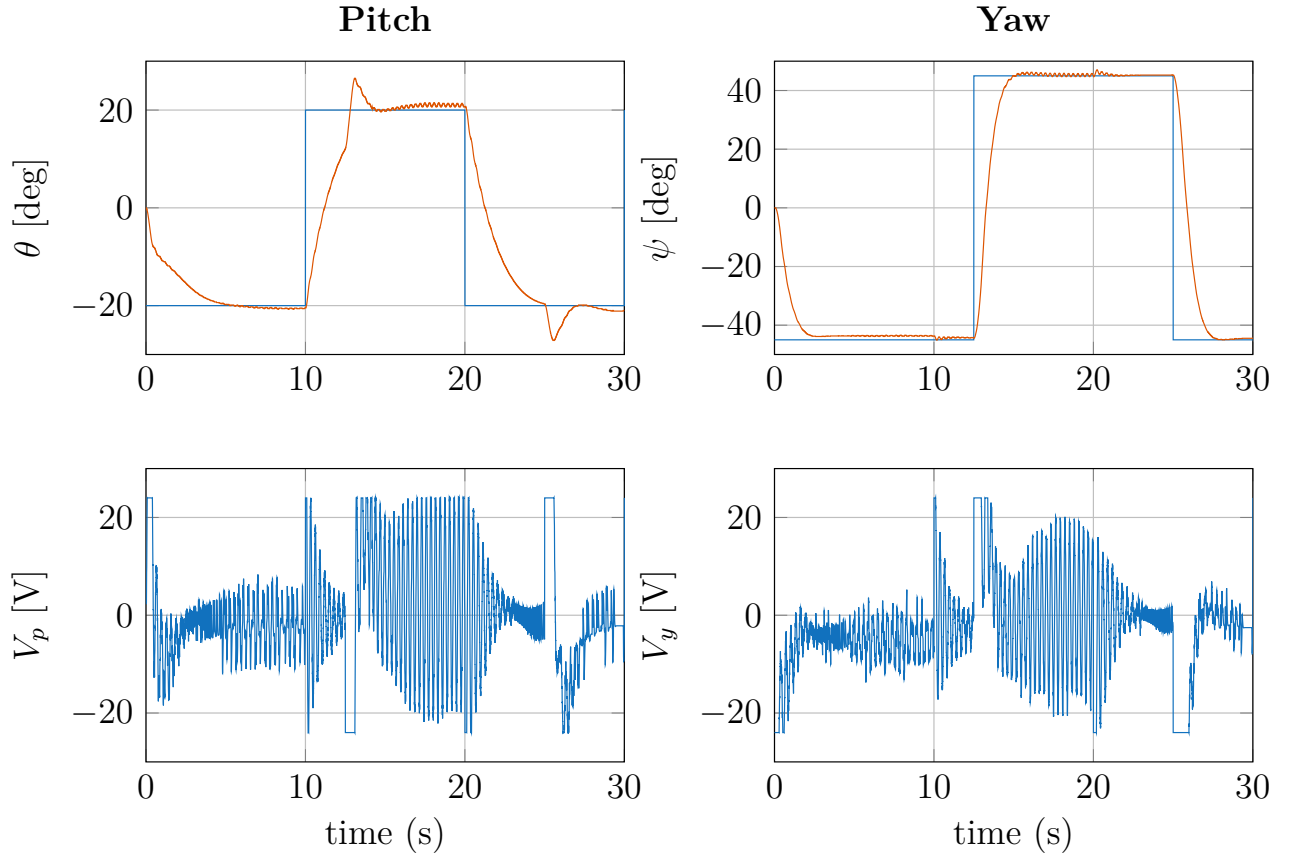


Figure 31: Results of the LQI simulation on the real plant with the input matrix $R_a = \begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$

We also tested the regulator with the gain eq. (16) with 2 sine reference signals with the same amplitude and frequency to observe the output and the command inputs. We chose for both references sine functions, because the sine function is continuous everywhere and the discontinuities cause big swings in the command inputs, swings that reach often the saturation point.

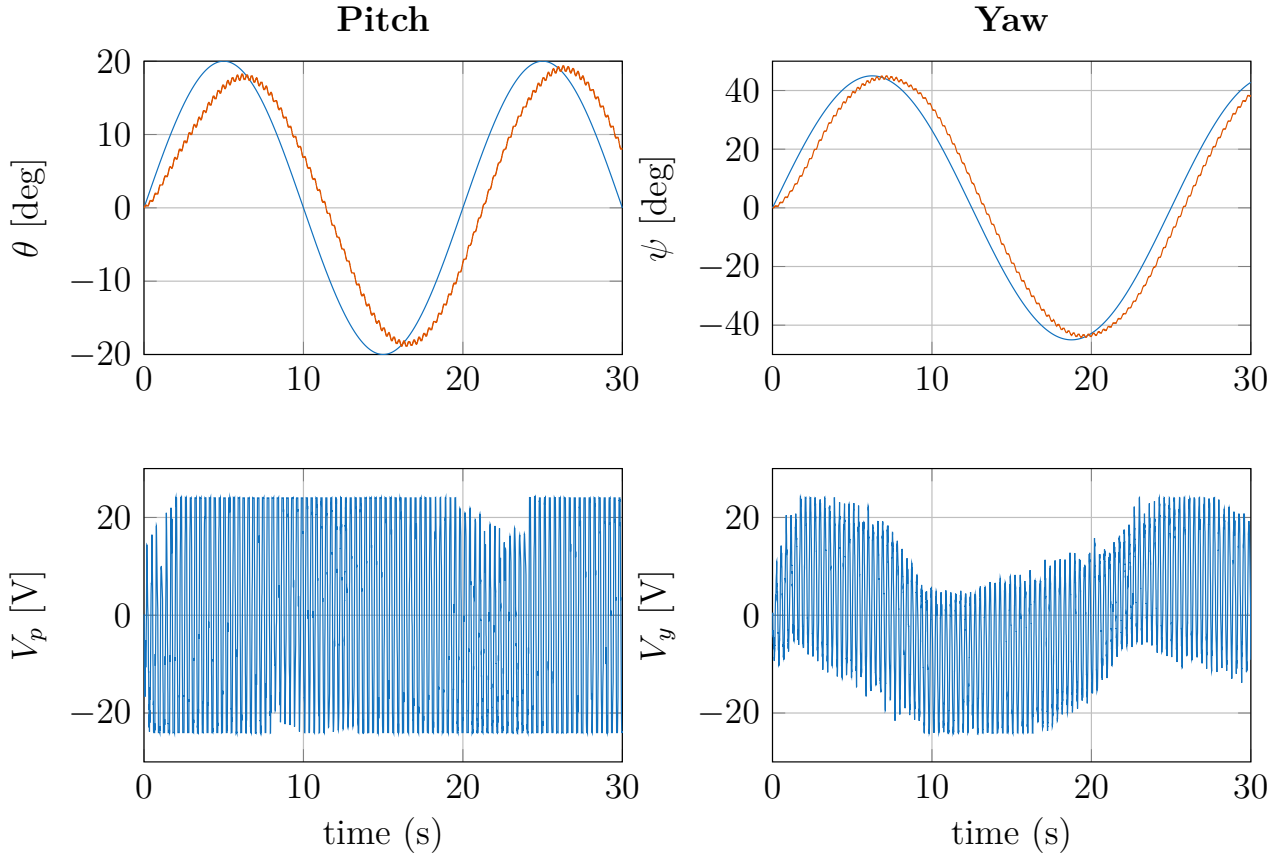


Figure 32: Results of the LQI simulation on the real plant with sine signal references and the controller with the gain from (eq. (16))

Both references are tracked with a phase delay and a diminished amplitude, but the command inputs still have oscillations with high amplitudes and frequencies.

4.4 PP controller based on a modified linear model

A possible cause for the high sensitivity in the pitch motion and the high magnitude of the inputs for the PP and LQI controllers might be because of calculating the linear model eq. (7) in the state $x^* = [0 \ 0 \ 0 \ 0]^T$, which implies that $\theta^* = 0 \text{ deg} = 0 \text{ rad}$. But we are testing the model for the pitch at $\theta^* = \pm 20 \text{ deg} = \pm 0.3491 \text{ rad}$ and $\psi^* = 45 \text{ deg} = \frac{\pi}{4} \text{ rad}$. A good proposal is to recalculate the model starting from the nonlinear model eq. (3) by using the `trim` and `linmod` commands in MATLAB around the state $x^* = [0.3491 \ 0 \ 0 \ 0]^T$. Because of the integrator in the $\psi(t)$, it won't stabilize in open-loop at a certain value (we would rather obtain a ramp response), so we'll keep it's setpoint value at 0:

```

1 %%
2 theta = deg2rad(20);
3 xref = [theta, 0, 0, 0]';
4 [xstar, ustar, ystar, ~] = trim('nonlinear_model_pin', xref, [], [], ones(4,1)
5 ,[],[])
6 [A, B, C, D] = linmod('nonlinear_model_pin', xstar, ustar);
7 aero2_ss = ss(A,B,C,D)
8
9 save('PPlin_20.mat', "A", "B", "C", "D", "aero2_ss", "ustar", "ystar", "xstar"
10 );
11 disp('A = ')

```

```

11 disp(A);
12 disp('B = ')
13 disp(B)
14
15 % open-loop poles:
16 eig(A)
17
18 % controllability
19 R = ctrb(A, B)
20 disp(rank(R))
21
22 % observability
23 O = obsv(A, C)
24 disp(rank(O))

```

From which we obtain this stationary points:

$$\begin{aligned}
 x^* &= \begin{bmatrix} 0.3491 \\ 0 \\ 0 \\ 0 \end{bmatrix} \in \left(\begin{bmatrix} -\theta_s \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} +\theta_s \\ 0 \\ 0 \\ 0 \end{bmatrix} \right) \\
 u^* &= \begin{bmatrix} 39.96 \\ 20.8971 \end{bmatrix} \notin \left(\begin{bmatrix} -24 \\ -24 \end{bmatrix}, \begin{bmatrix} +24 \\ +24 \end{bmatrix} \right)
 \end{aligned} \tag{17}$$

which shows again that the linear model is calculated around an equilibrium point, whose input u^* does not respect the physical boundaries of the pitch motion.

The linear model calculated is:

$$\begin{aligned}
 A &= \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0.5071 & 0 & -0.0858 & 0 \\ 0 & 0 & 0 & -0.08063 \end{bmatrix}, \\
 B &= \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0.02317 & 0.009889 \\ -0.02243 & 0.04289 \end{bmatrix}, \\
 C &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \\
 D &= \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}
 \end{aligned} \tag{18}$$

which is both controllable and observable, and it has these open-loop poles:

$$\Lambda_o = \left\{ \begin{array}{l} 0, \\ -0.0806, \\ -0.2721, \\ 0.1863 \end{array} \right\}$$

which is not stable, because of $\lambda_1 = 0$ and $\lambda_4 = 0.1863 > 0$, but it is stabilizable, because the system is controllable.

And now we're calculating the K gain based on this linear model by using the Pole Placement method with the desired poles eq. (9).

The `place` command was succesful in MATLAB and we obtained the gain:

$$K = \begin{bmatrix} 307.3546 & -70.4649 & 142.8574 & -32.9858 \\ 160.7313 & 165.1040 & 74.7074 & 77.2878 \end{bmatrix} \tag{19}$$

This gain stabilizes the model in the desired closed-loop poles and we're testing it on the QUARC simulator and the real plant with the tanh approximation on these reference tracking signals:

- pitch angular position θ : step signal with the amplitude of $\theta^* = 20$ deg and a 1 s delay, because we calculated the linear model around this setpoint for the pitch motion.
- yaw angular position ψ : square-wave signal with the amplitude $\psi^* = 45$ deg and the frequency $f = 0.04$ Hz, suitable to detect the sensitivity of the controller⁴.

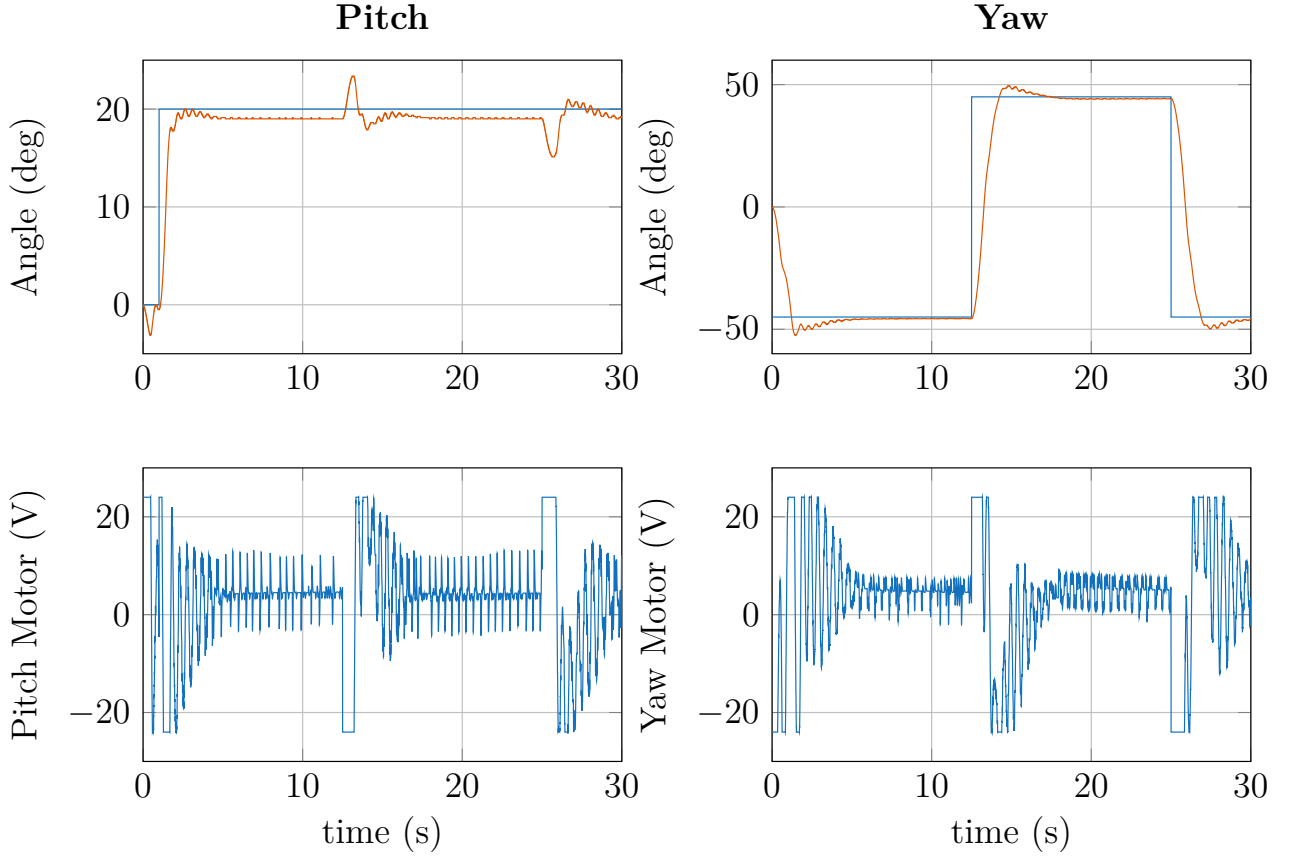


Figure 33: Results of the eq. (19) PP controller on the QUARC Simulator

⁴To distinguish the 2 disturbances that determine the sensitivity, we chose to mark the overshoot with positive (+) percentages and the undershoot with negative (-) percentages

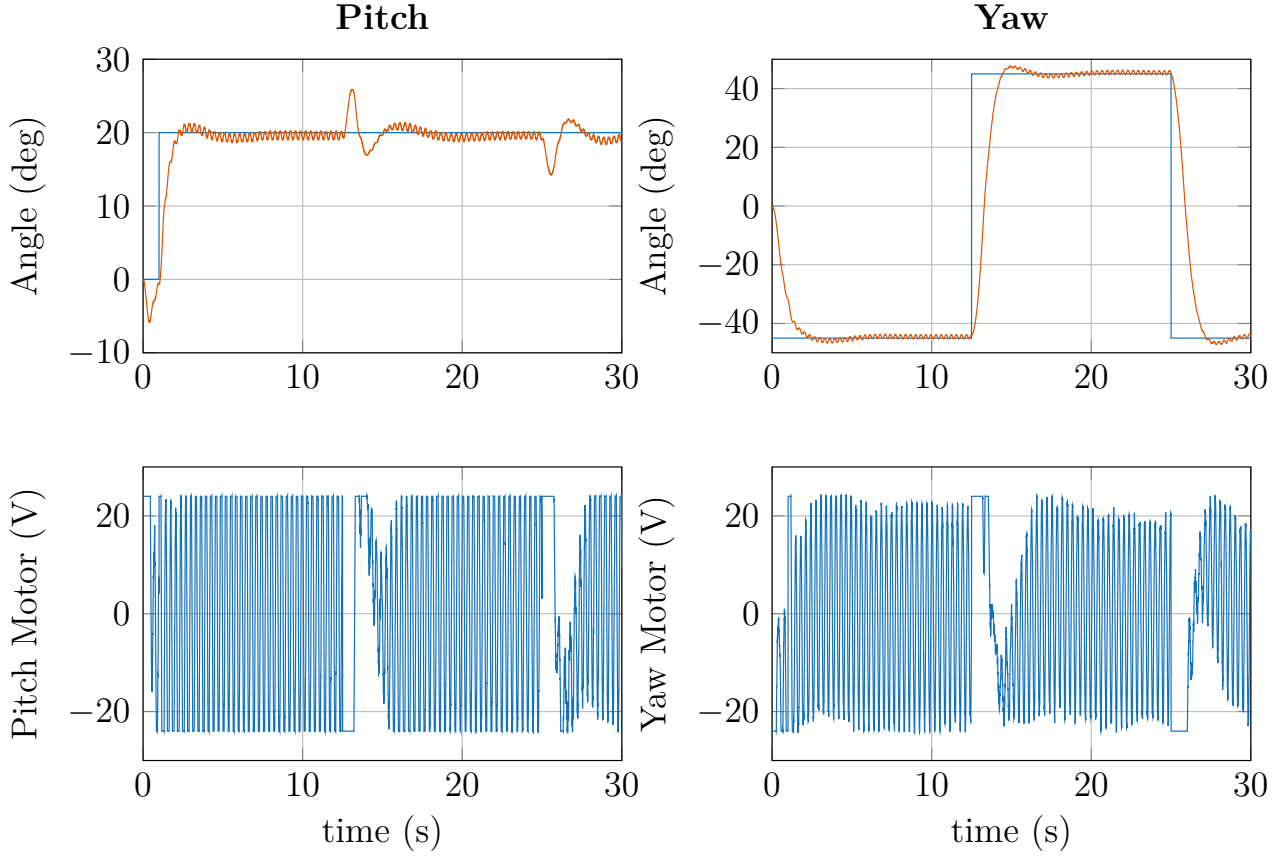


Figure 34: Results of the eq. (19) PP controller on the real plant

Pitch Axis (θ)		
Parameter	Simulator	Real Plant
Relative steady state error (ϵ)	$\approx 5\%$	$\approx 1.8\%$
Overshoot (σ)	$\approx 5.2\%$	$\approx 5.4\%$
Peak time (t_p)	1.6 s	1.9 s
Coupling overshoot/undershoot (σ')	$\approx +23\% / -19\%$	$\approx +31.85\% / -27.38\%$
Yaw Axis (ψ)		
Parameter	Simulator	Real Plant
Relative steady state error (ϵ)	$\approx 2\%$	$\approx 1\%$
Overshoot (σ)	$\approx 12.54\%$	$\approx 4.7\%$
Peak time (t_p)	2.3 s	2.41 s

Table 3: Performance comparison of the eq. (19) Controller across the three tests

Looking at the performances of section 4.4, compared to section 4.1, there are no significant differences between the 2 approaches, despite the coupling overshoot/undershoot being slightly lower. Moreover, the command inputs have the same magnitude and frequency as the PP controller computed based on the eq. (7) and the outputs have oscillations with higher amplitudes in the steady in the case of the eq. (19) controller.

4.5 Conclusions

The conclusion for all 3 controllers is that the performances are not excellent, as the commands are calculated without considering the constraints, but these are forced on the command after the calculations.

A better approach is to use a controller that considers the system's constraints in computing the commands, which leads us to the Predictive Control methods.

5 Predictive Control Methods

5.1 Introduction to Predictive Control

The biggest disadvantage of the classical control methods is that it doesn't tackle the physical constraints of the real plants in designing the command, but rather the computed input is clipped in order to fit the physical requirements.

This is how we ensured in the previous experiments that the command inputs remained in the $[-24 \text{ V}, +24 \text{ V}]$: by adding a **Saturation** block or a **MATLAB Function** block, that approximates the **Saturation** through a linear combination of tanh functions. The problem is that the command was computed in order to follow the reference signal and reject the disturbances without losing the stability of the plant, but often it didn't respect the constraints, so we had to filter it, which implicitly deteriorated the performances.

The idea of this predictive control approach is to build a controller that **optimizes** the command input after a cost function (like the solving the linear quadratic problem) considering both the model⁵ and the constraints.

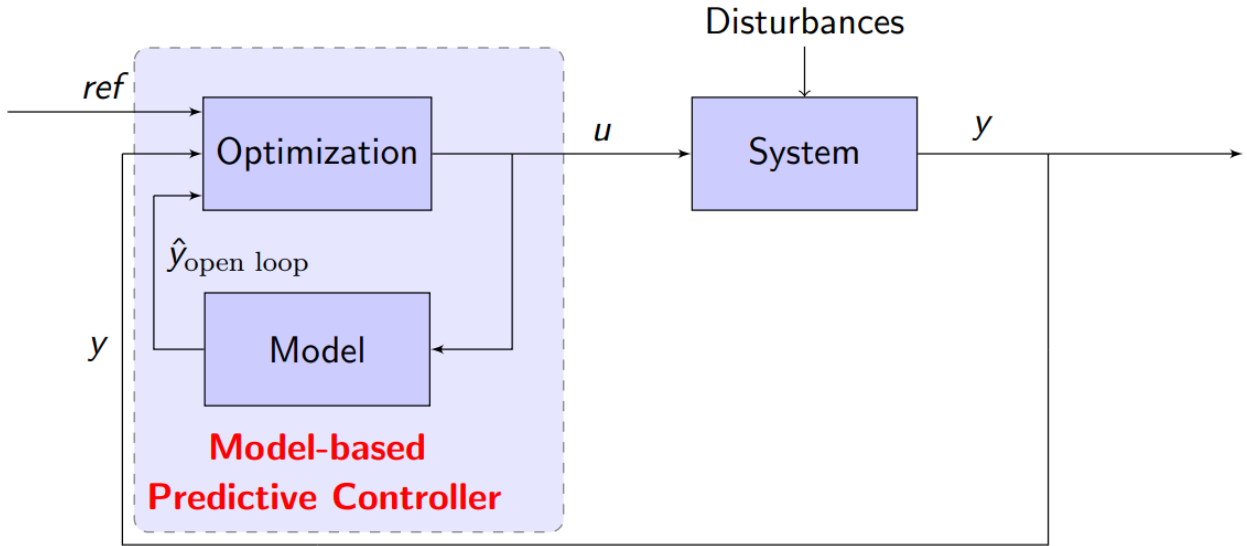


Figure 35: Closed-loop scheme with a Model-based Predictive Controller based on the constrained model, adapted from [4]

One of the first problems is that it consists of solving a constrained optimization problem that includes states and commands at *any time*, which implies that we would have a problem with infinite constraints, which is impossible to compute. Because of this problem, we have to limit ourselves to a finite receding horizon and we have to discretize the system in order to properly quantify the finite horizon.

⁵It considers the model only if it is a Model-based controller. Currently, there are approaches that abandon the model, like the Data-Driven Predictive Controller

5.2 Linear Model-based Predictive Control

I start this approach with the linear discrete model, that should approximate properly the plant, the standard form for this Model-based Predictive Control (MPC) being:

5.3 Nonlinear Model-based Predictive Control

We'll simulate first an MPC problem by using the linear model eq. (7), discretized by using eq. (20) with $T_s = 0.02$ s $\implies N_{sim} = \frac{t}{T_s} = 1.500$ samples for reaching a certain setpoint, defined by:

- $\theta^* = -20$ deg
- $\psi^* = 45$ deg

these parameters:

- $N_{pred} = 20$ samples;
- $Q_{MPC} = T_s Q_{LQR}$;
- $R_{MPC} = T_s R_{LQR}$ ⁶;
- $P_{MPC} = 100Q_{MPC}$ - the high coefficient of the terminal cost assures that the system does reach its final point.
- CasADi solver: `ipopt`

and these constraints:

- state constraint: $-0.89 \leq \theta_k \leq 0.89$
- **input constraint:** $-24 \leq u_k \leq 24$
- input variation constraint: $-5 \leq u_k - u_{k-1} \leq 5$

⁶The MPC problem derives from the LQR problem, on which we add the constraints on the state x , input u and the variation of the input Δu , except we implemented the LQR controller on the continuous model and the MPC on a discrete model. Because the LQR controller had the best performances in terms of the output performances compared to its input magnitudes, we can take those Q and R matrices and we multiply them by the sampling time.

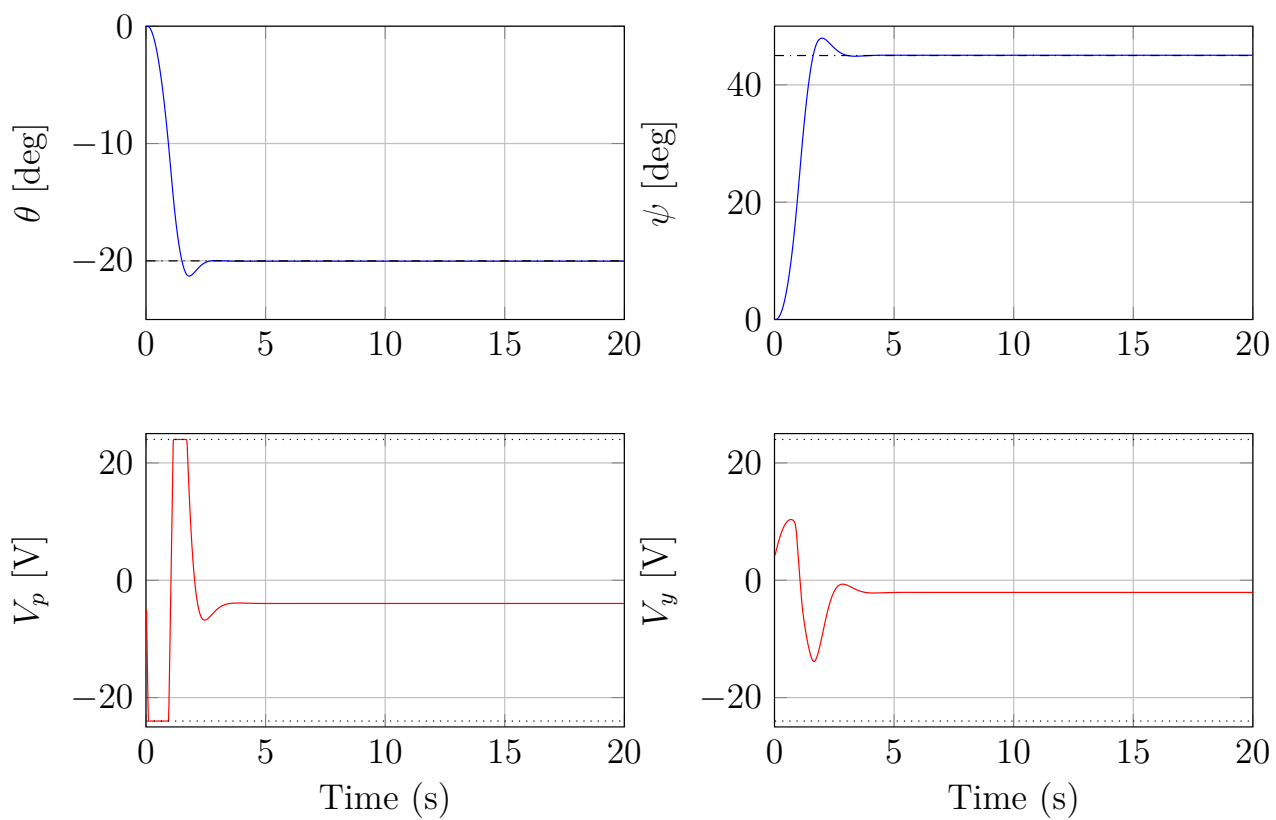


Figure 36: v

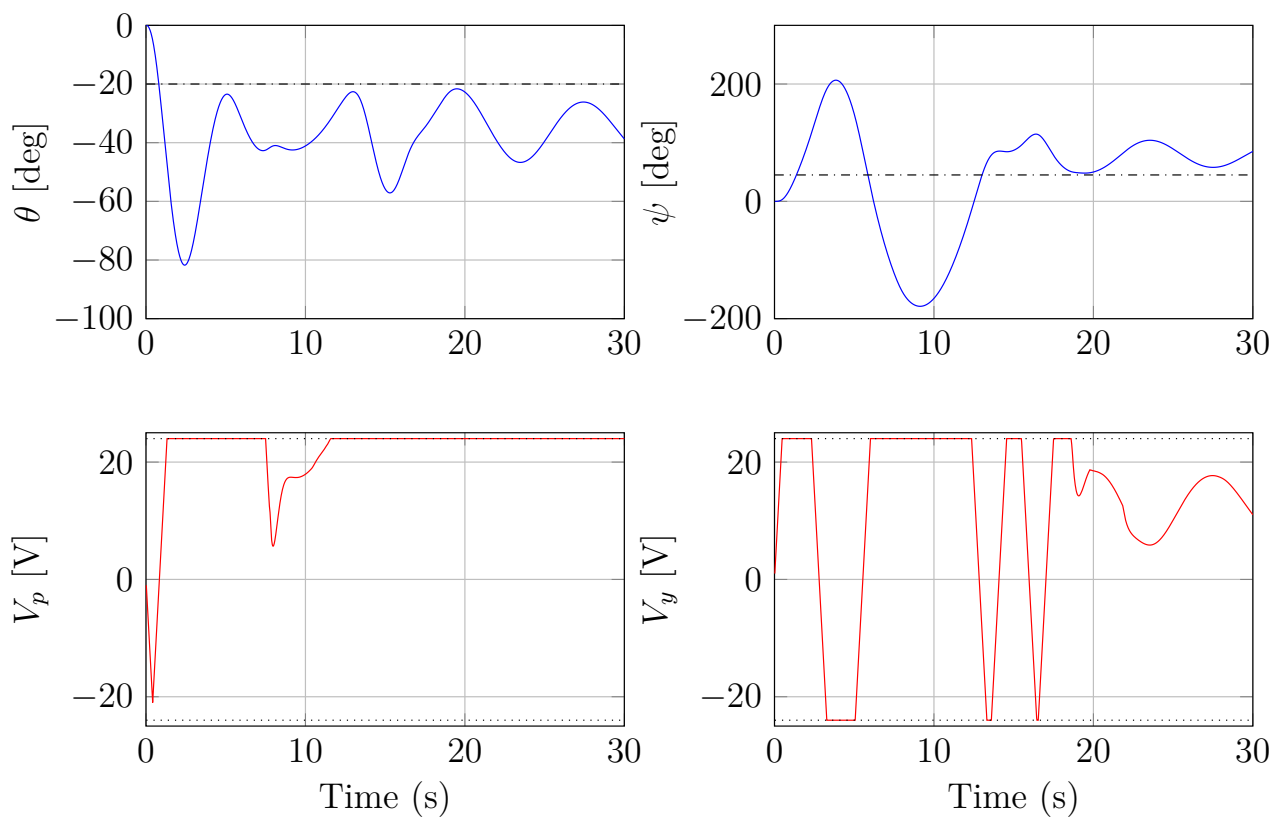


Figure 37: c

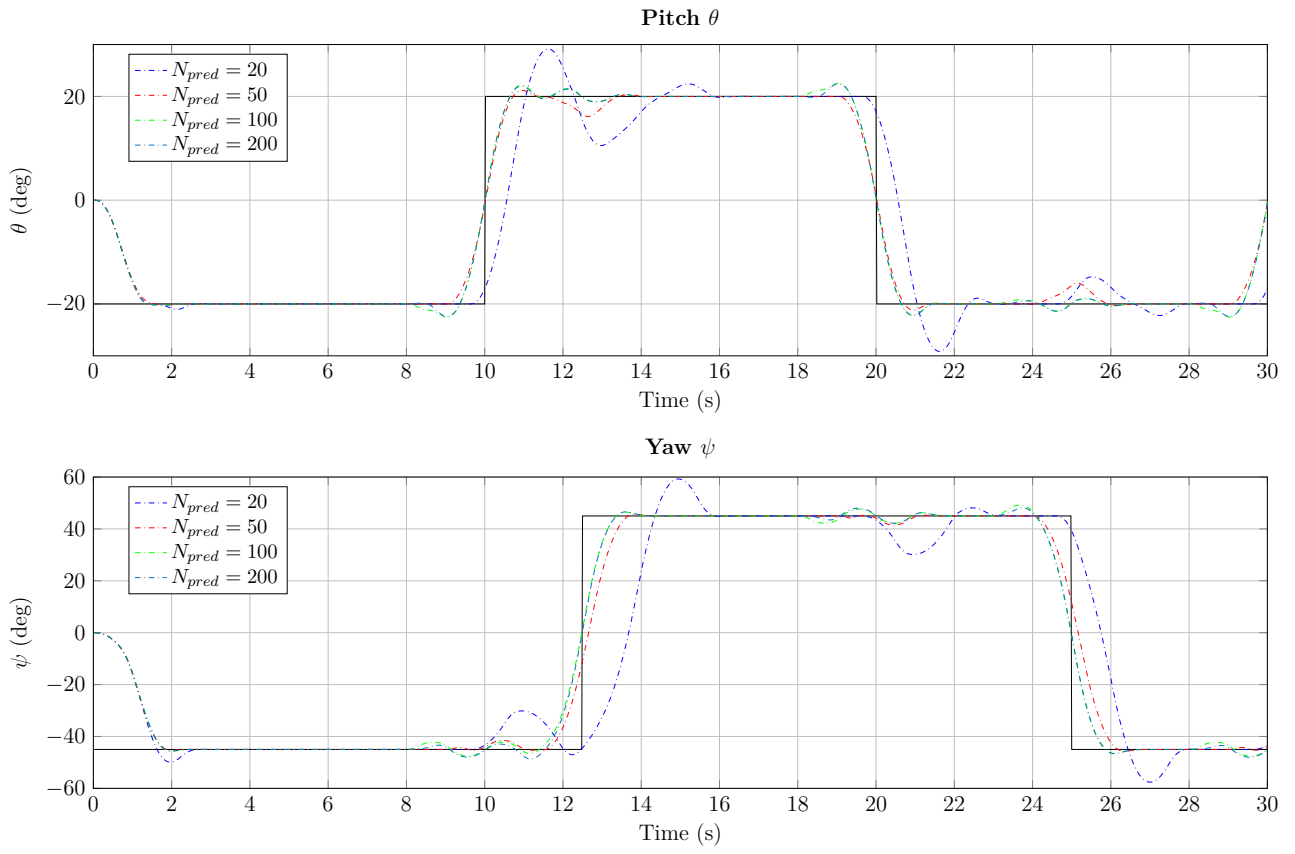


Figure 38: c

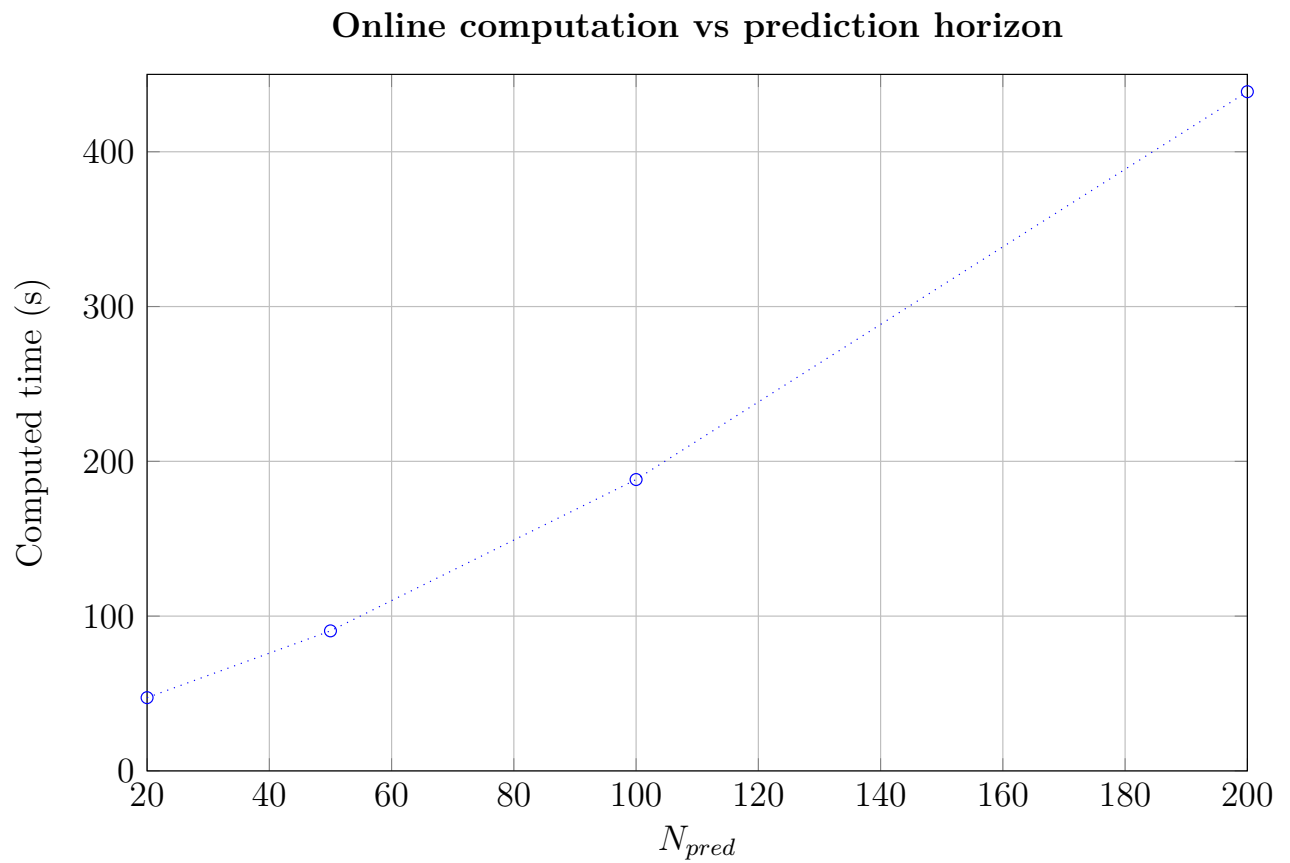


Figure 39: c

5.4 MPC Implementation

From [5]

```

1 %% Generate CasADi function for Simulink
2 F = solver.to_function('mpc_solver', ...
3     {xinit, xref}, {u(:,1)}, ...
4     {'x0','xref'}, {'u_opt'});
5
6 cg_options = struct;
7 cg_options.casadi_real = 'real_T';
8 cg_options.casadi_int = 'int_T';
9 cg_options.real_min = num2str(realmin('double'));
10 cg_options.with_header = true;
11 cg = casadi.CodeGenerator('mpc_solver', cg_options);
12 cg.add_include('simstruc.h');
13 cg.add(F);
14 cg.generate();
15 fprintf('C code generated \n');
16
17 %% Compile MEX file
18 fprintf('\nCOMPILING S-FUNCTION\n');
19 mex('mpc_solver_wrapper.c', 'mpc_solver.c');

```

We generated .c files from the simulation script and incorporated them in an S-Function Block.

We also tested multiple sampling times for the ode1 integration algorithm of Simulink.

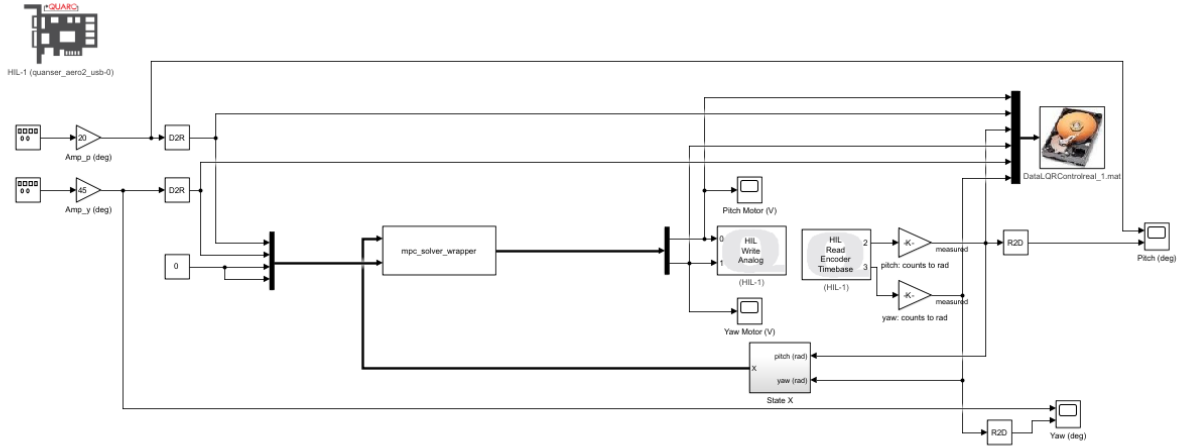


Figure 40: Simulink scheme of the implementation on the QUARC Simulator and the real plant

The MPC tested and implemented had the eq. (7) model discretized with $T_s = 0.2$ seconds with the eq. (20) method, the same reference signals for the pitch and yaw angles and:

- optimization problem of targeting a setpoint (current references) $\implies$ no prediction of reference changes

- $N_{pred} = 20$ samples

- $Q = \begin{bmatrix} 10.000 & 0 & 0 & 0 \\ 0 & 5.000 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 50 \end{bmatrix}$

- $R = 0.1I_2$
- $P = 10Q$
- no constraints on $\Delta u_k = u_k - u_{k-1}$
- solver: `qrqp`⁷ $\Rightarrow$ very sensitive at the choices of the MPC parameters (if Q is too big and R too small, it crashes)

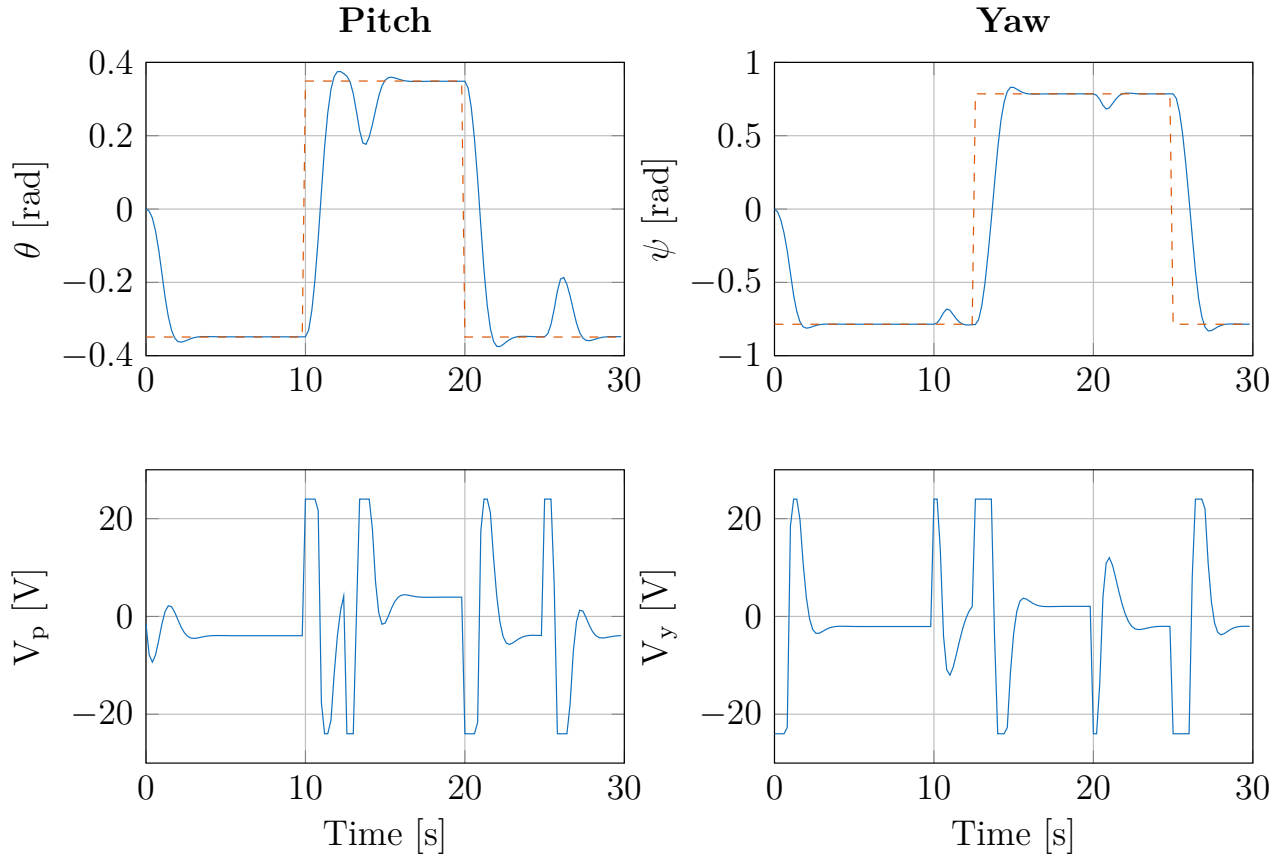


Figure 41: Linear simulation of the MPC controller

⁷through CasADi

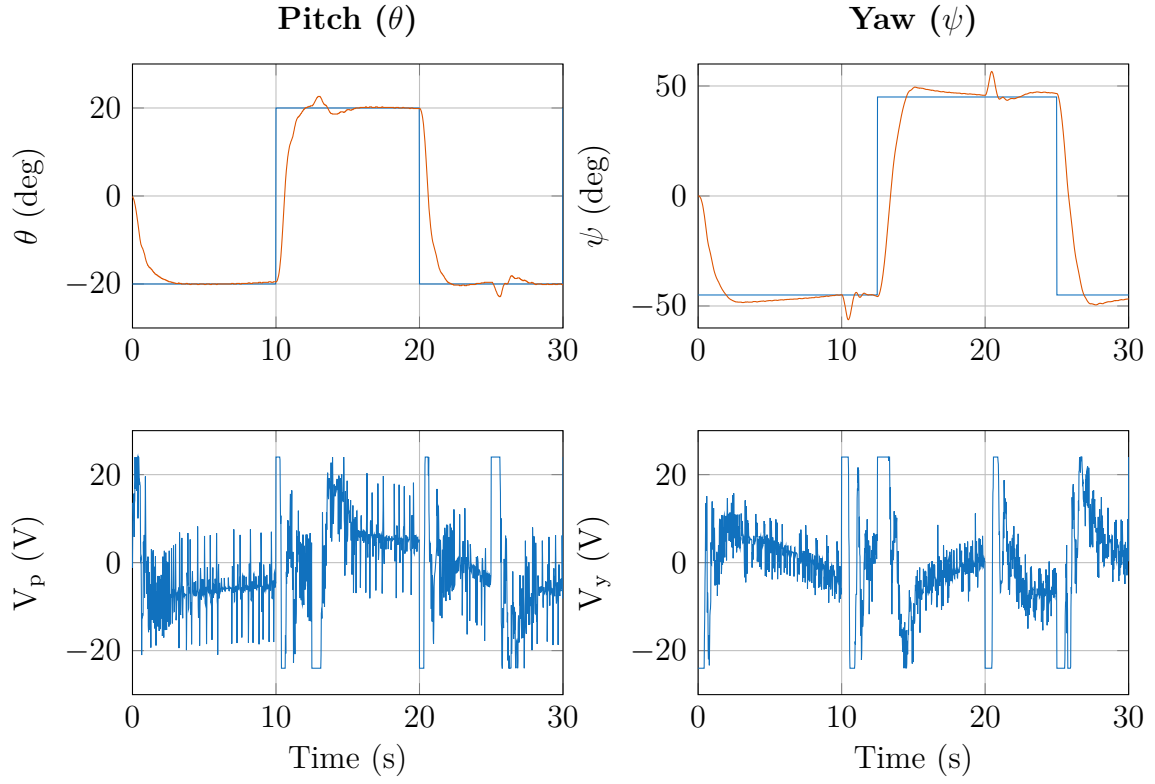


Figure 42: Results of the MPC controller on the QUARC Simulator

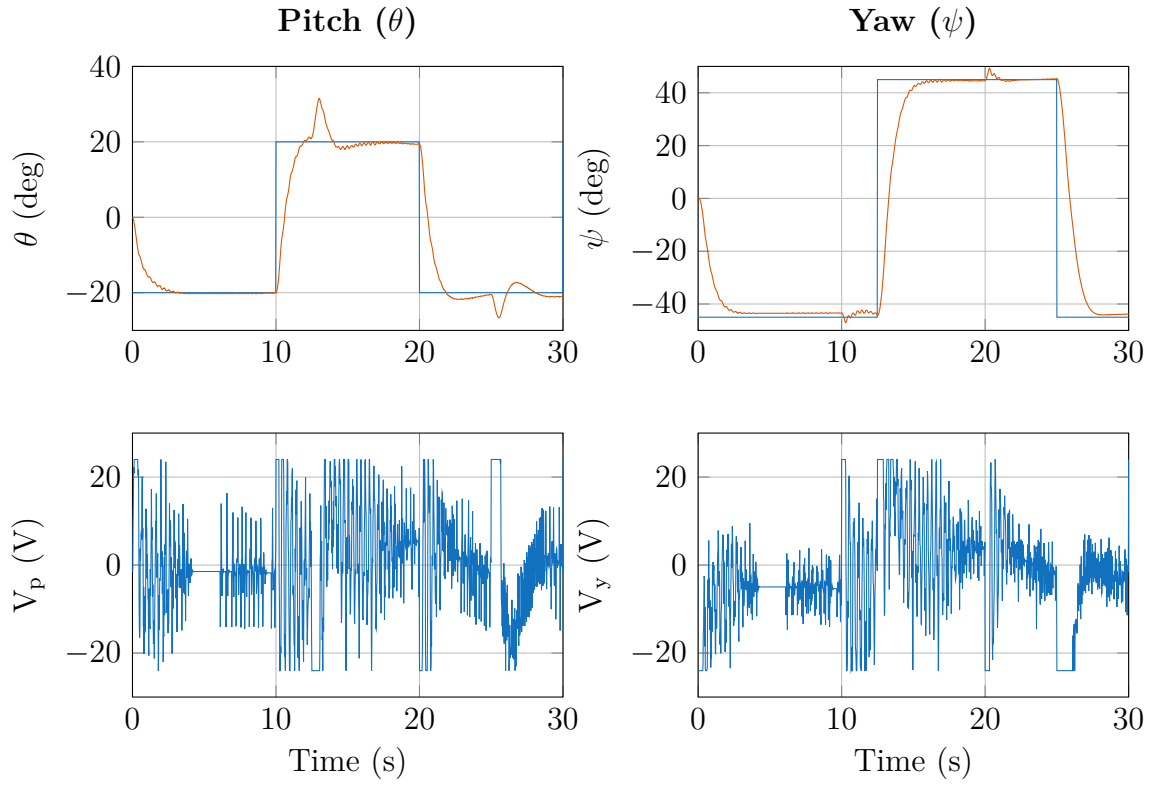


Figure 43: Results of the MPC Controller on the real plant

A Discrete model

One of the main problems with the model used is that the equilibrium points of both linearized models based on eq. (3) do not respect the physical constraints, which is why I decided to grid the state equilibrium points of the nonlinear model based on the command inputs between $[-24 \text{ V}, +24 \text{ V}]$ in order to visualize the possible equilibrium points that respect all of the constraints. So we plotted each component of the x^* obtained from `[xstar, ~, ~, ~] = trim('nonlinear_model_pin', zeros(4,1), uref, [], [], ones(2,1), [])`; where u_{ref} represents 400 equidistant points in the $[-24 \text{ V}, +24 \text{ V}] \times [-24 \text{ V}, +24 \text{ V}]$ grid and kept only the points that respect the state constraint on $\theta \in [-\theta_s, +\theta_s]$:

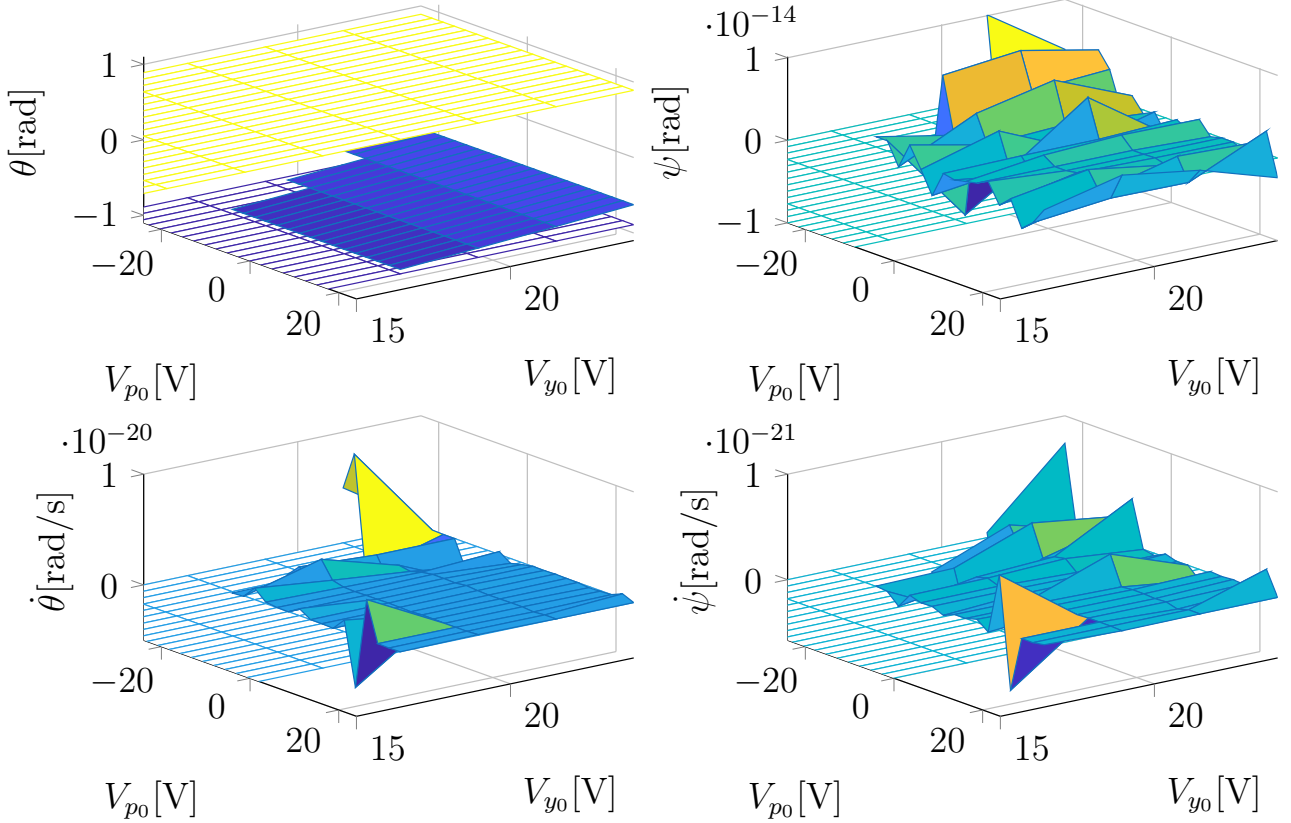


Figure 44: Stationary states x^* of the nonlinear model, whose u^* respects the physical constraints, with emphasizing the $\pm\theta_s$ limits and the 0 reference for the other 3 states

fig. 44 emphasizes that we can only build a linear model around $\theta \in [-47.62 \text{ deg}, -35.82 \text{ deg}] \approx [-0.8312 \text{ rad}, -0.6251 \text{ rad}]$ and, for $\theta^* = -35.82 \text{ deg} \implies V_{y0} = +24 \text{ V}$. Moreover, the yaw motion doesn't change at all from its initial state, as we started each simulation for each input from $x_0 = \mathbf{0}_{4 \times 1}$, as the changes are happening on a scale of 10^{-14} , and both angular speeds stabilize at 0 rad/s.

For tracking a setpoint, we'll compute the linearization around the state $x_{ref} = [-0.7071 \text{ rad} \ 0 \ 0 \ 0]^T$ and we'll discretize the linear model by using the explicit Euler formula:

$$\dot{x}(t) \approx \frac{x_{k+1} - x_k}{T_s} \quad (20)$$

where the sampling time of the QUARC Aero 2 is, according to the lab indicators [2], $T_s = 0.002 \text{ s}$.

We obtained the linear model:

$$\begin{aligned}
A_d &= \begin{bmatrix} 1 & 0 & 0.002 & 0 \\ 0 & 1 & 0 & 0.002 \\ -0.002052 & 0 & 0.9998 & 0 \\ 0 & 0 & 0 & 0.9998 \end{bmatrix} \\
B_d &= \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 4.634 \cdot 10^{-5} & 1.978 \cdot 10^{-5} \\ -4.485 \cdot 10^{-5} & 8.577 \cdot 10^{-5} \end{bmatrix} \\
C_d &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\
D_d &= \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}
\end{aligned} \tag{21}$$

which is both controllable and observable and it has these open-loop poles:

$$\Lambda = \left\{ \begin{array}{l} 1, \\ 0.9998, \\ 0.9999 \pm 0.002i \end{array} \right\}$$

which has an integrator and, while it is controllable, it requires a lot of samples to notice proper results (the previous simulations ran for $t = 30 \text{ s} = NT_s \implies N = \frac{t}{T_s} = 15.000$ samples), which implies a huge computational time.

Moreover, this model has its stable point at a really big θ , close to one of the saturation points, which is problematic for realistic practises, because this is a functionality at its physical limits, both as the pitch angle, and the stable inputs.

Bibliography

- [1] *Quanser Aero 2 User Manual*. V2.1. Quanser Inc. 119 Spy Court Markham, Ontario, Canada, 2022.
- [2] Ionela Prodan. *Classical feedback control for an inverted pendulum and a 2DOF helicopter*. AU427 Lab Session. 2026.
- [3] Dennis S. Bernstein. *Matrix Mathematics*. 2009, pp. 77–164.
- [4] Sorin Olaru. *(Model-based) Predictive Control — Lecture Notes*. Master ATSI, Centrale-Supélec. 2022.
- [5] Ionela Prodan and Cong Khanh Dinh. *Model Predictive Control Design and implementation for a DC Servo Motor*. AU427 Lab Session. 2026.